2023 年全国大学生信息安全竞赛

作品报告

作品名称：	《AndroFuzz-面向安卓 Native 系统服务的漏洞挖掘技术》
电子邮箱：	18862006333@163.com
提交日期：	2023 年 8 月 9 日

填写说明

1. 所有参赛项目必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛项目（或方案）。

2. 作品报告采用A4纸撰写。除标题外，所有内容必需为宋体、小四号字、1.5倍行距。

3. 作品报告中各项目说明文字部分仅供参考，作品报告书撰写完毕后，请删除所有说明文字。(本页不删除)

4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。

5. 为保证网评的公平、公正，作品报告中应避免出现作者所在学校、院系和指导教师等泄露身份的信息。

目　录

摘要 1

第一章 作品概述 4

1.1 研究背景 4
1.1.1 Android 系统服务介绍 6
1.2 研究现状 8
1.2.1 安卓系统安全 8
1.2.2 模糊测试与安卓系统服务 9
1.3 作品主要工作 11
1.4 特色描述 12
1.4.1 漏洞挖掘覆盖率全面 12
1.4.2 自动化提取与检测 13
1.4.3 多版本适用性和易调整性 13
1.5 应用前景 14
1.5.1 适应多厂商环境和商业市场广阔 14
1.5.2 安全合规性自动化验证平台 15
1.5.3 Android ROM 定制 15
1.6 小结 16

第二章 作品设计及实现 17

2.1 相关技术及理论 17
2.1.1 模糊测试技术(Fuzzing) 17
2.1.2 Binder 通信机制 18
2.1.3 Native 系统服务接口调用与 Android 接口定义语言(AIDL) 19
2.1.3.1 Native 服务接口调用 19
2.1.3.2 Android 接口定义语言（AIDL） 22
2.1.4 Android flash 23
2.1.5 HWAddressSanitizer 工具 24
2.2 设计方案与核心技术 25
2.2.1 整体方案 25
2.2.2 接口文件收集方案 26
2.2.2.1 记录编译命令 26
2.2.2.2 收集系统服务接口相关的文件 28
2.2.3 接口模型提取方案 29
2.2.3.1 接口模型预处理 29
2.2.3.1.1 cc1 编译命令提取 29
2.2.3.1.2 Clang 插件及使用 30
2.2.3.2 目标接口模型提取 32
2.2.4 依赖推断方案 37
2.2.4.1 变量依赖 37
2.2.4.2 服务接口依赖 40

2.3 作品实现 42
2.3.1 Fuzzer 引擎 42
2.3.1.1 Fuzzer 流程概要 42
2.3.1.2 事务生成器 43
2.3.1.3 执行器 44
2.3.2 Manager 模块 46
2.3.2.1 检测设备环境准备 46
2.3.2.2 Fuzzer 运行 47
2.3.2.3 Fuzzer 工作总结 47
2.4 小结 48
第三章 作品测试与分析 49
3.1 测试方案 49
3.1.1 生成 Native 系统服务接口模型测试方案 49
3.1.2 依赖关系推断测试方案 50
3.2 测试环境及设备 51
3.3 功能测试结果及分析 51
3.3.1 Native 系统服务接口提取与依赖推断 51
3.3.2 事务接口模型 53
3.4 作品测试实验结果 54
3.5 案例分析 56
3.5.1 漏洞 1 (new_capacity overflow) 56
3.5.2 漏洞 2 (heap-buffer-overflow) 57
3.5.3 漏洞 3 (CHECK failure) 59
3.5.4 漏洞 4 (CHECK failure) 60
3.6 小结 62
第四章 创新性说明 63
4.1 研究方向开创性 63
4.2 技术实现巧妙性 64
4.3 系统实战可行性 64
4.4 小结 65
第五章 总结与展望 66
5.1 总结 66
5.2 展望 67
参考文献 68
附录 A Native 系统服务接口依赖图 73
附录 B 作品总体框架大图 74

图 目 录

图 1- 1 Google 更新 Android 各版本份额 6
图 1- 2 Google 提供的 Android 系统架构图 7
图 1- 3 近五年 SCI 收录关于 Android 系统安全论文量比图 9
图 1- 4 作品总体框架 12

图 2- 1 模糊测试流程图 18
图 2- 2 Binder 通信机制架构 19
图 2- 3 Native 系统服务调用程序 20
图 2- 4 AIDL 潜在漏洞类型 22
图 2- 5 作品整体方案图 26
图 2- 6 Clang AST 插件遍历的节点 31
图 2- 7 onTramsact 函数的 swich-case 语句 32
图 2- 8 顺序变量源码样例 33
图 2- 9 条件变量源码样例 35
图 2- 10 循环变量源码样例 36
图 2- 11 返回值变量源码样例 36
图 2- 12 类型定义源码样例 38
图 2- 13 系统服务接口 JSON 格式图 38
图 2- 14 部分服务接口依赖图 42
图 2- 15 Fuzzer 流程图 43

图 3- 1 Native 系统服务接口数据柱状图 52
图 3- 2 事务接口模型数据图 53
图 3- 3 IDrm 系统服务崩溃错误信息及调用栈 56
图 3- 4 readVector 源码 57
图 3- 5 StatsService 系统服务崩溃调用栈 57
图 3- 6 StatsService 系统服务崩溃调用路径 58
图 3- 7 CHECK failure 错误调用栈信息 60
图 3- 8 MediaRecorderClient 源码 60
图 3- 9 CHECK failure 错误调用栈信息 61
图 3- 10 clearSecureUserId 源码 61

表 目 录

表 2- 1　算法 1 服务接口相关文件收集 28
表 2- 2　算法 2 事务间变量推断器 39
表 2- 3　算法 3 接口间依赖推断 41
表 2- 4　算法 4 自我约束（self_constraint)处理 44
表 2- 5　算法 5 执行器处理 45

表 3- 1　发现漏洞统计列表 55

表 5- 1　报错日志漏洞信息表 66

术语表

名称	解释说明
AndroFuzz	用于检测安卓 Native 系统漏洞的模糊测试工具。
Android Native 系统服务	是 Android 操作系统中提供系统级服务、硬件设备抽象与控制、系统资源管理和安全性管理的核心组件。
AOSP	全称是 Android Open Source Project，它是 Android 系统的开源项目。
Clang AST	它是一个由 Clang 编译器生成的数据结构，它代表源代码中的语法和语义结构，常用于编译器优化、源代码分析和其他工具开发中。
Fuzzer	是一种自动化的软件测试技术，它通过向系统或程序输入大量随机或特意构造的数据，触发异常行为，用于发现编程错误和安全漏洞。
API	全称应用程序接口（Application Programming Interface），是一组定义了如何通过函数、数据结构等进行交互的规范，使得软件间可以互相通信、协同工作，为开发者在构建和集成软件应用中提供便利。
AST	抽象语法树（AST）是源代码的抽象语法结构的树状表现形式，其中每个节点都代表了构造源代码的一部分，它在编译器或其他语法分析工具中起着核心作用，用于理解程序的结构和语义。
IPC 进程间通信	它是指在独立的系统进程之间共享信息和数据的机制，包括但不限于管道、消息队列、共享内存、信号量和套接字等方法。
代理(Proxy)	是一种网络服务，它接收来自客户端的请求，然后代替客户端向目标服务器发送请求，并将服务器的响应返回给客户端，常用于数据过滤、网络连接共享、安全防护等场景。
存根对象(Stub)	它是在分布式计算或者软件测试中，用来模拟远程服务、服务组件或模块功能的一个对象，它接收客户端的调用并响应预先设定的行为或数据，从而方便测试或者实现系统的解耦。
AIDL	全称 (Android Interface Definition Language) ,是一种 Android 特有的接口定义语言，用于让 Android 应用程序的组件能够相互通信，尤其在多进程环境下进行跨进程通信(IPC)。
IDrm	全称为 (Interface to Digital Rights Management) ,是 Android 中的一个接口，它负责管理数字内容的权限，保证数字媒体内容的安全传输和播放。
数据序列化/反序列化	数据序列化是将数据结构或对象状态转换为可以存储或传输的格式的过程，而反序列化则是将这种格式的数据恢复为其原始的数据结构或对象状态。

摘要

安卓（Android）系统作为全球最受欢迎的操作系统，其开源的特性使得安卓成为了众多手机制造商的首选。目前安卓系统在全球市场占据主导地位，全球四分之三的智能手机都运行安卓系统。这当中，安卓系统安全尤其值得我们重点关注。安卓系统中，**Android Native 系统服务为用户应用提供必要支持和核心功能，包括系统级服务、硬件设备的抽象和控制、系统资源管理以及安全性管理。而 Android Native 系统服务的漏洞可以引发多种危害：数据泄漏、设备性能下降等。为了保证手机安全，发现其中的潜在漏洞是至关重要的。**

然而，根据当前国内外的研究现状，大多数提出对 Android 系统服务的测试仅对 Java 系统服务的针对性较强，并未针对性地测试 Native 系统服务。**即使工具如 ASFuzzer 和 BinderCracker 可以提取部分 Android native 系统服务接口，它们仍无法提取所有 native 系统服务接口，同时也缺乏精确的语义信息。**

本作品采用了基于生成的模糊测试来挖掘 Android Native 系统服务的漏洞。我们的方案如下：(1)通过对 AOSP 进行编译，并记录编译命令来收集所有与系统服务相关的文件，包括实名服务和匿名服务；(2)通过目标接口的 Clang AST 提取有效的语义信息并生成 JSON 格式的事务接口模型，并根据输入输出变量推断出事务间和接口间的依赖关系；(3)对 Fuzzer 进行设计和实现。**通过此方案，本作品具有优化支持 Android 最新版本 13、自动提取完整服务接口以及自动化测试与监控日志同步这三点开创性功能。**

我们共收集了 118 个 Android Native 系统服务,其中包括 41 个实名服务和 77 个匿名服务，此外还提取了 1313 个事务的接口模型。根据事务接口模型在 Android 13 版本的真机 Google Piexl 4xl 上进行模糊测试，并产生了大量的崩溃日志。通过对日志的分析，我们从新发现的 13 个漏洞中挑选出 4 个典型的新漏洞案例作出分析。实验结果证明了我们研发的 Fuzzer 在 Android 13 版本下的有效性，并且提出在未来更新的更高版本测试的可行性。

关键词： Android Native 系统服务;模糊测试;事务;依赖关系; Fuzzer

Abstract

Android system is the most popular operating system in the world, and its open source feature makes Android the first choice of many mobile phone manufacturers. Android currently dominates the global market, with three-quarters of the world's smartphones running Android. Among them, the security of the Android system deserves our special attention. In the Android system, Android Native system services provide necessary support and core functions for user applications, including system-level services, abstraction and control of hardware devices, system resource management, and security management. Vulnerabilities in Android Native system services can cause a variety of hazards: data leakage, device performance degradation, etc. In order to keep mobile phones safe, it is crucial to discover potential vulnerabilities in them.

However, according to the current research status at home and abroad, most of the tests proposed for Android system services are only highly targeted for Java system services, and do not specifically test Native system services. Even if tools such as ASFuzzer and BinderCracker can extract some Android native system service interfaces, they still cannot extract all native system service interfaces, and they also lack precise semantic information.

This work employs generation-based fuzzing to discover vulnerabilities in Android Native system services. Our solution is as follows: (1) Collect all files related to system services, including real-name services and anonymous services, by compiling AOSP and recording compilation commands; (2) extract effective semantic information through the Clang AST of the target interface and Generate the transaction interface model in JSON format, and infer the dependencies between transactions and interfaces according to the input and output variables; (3) Design and implement the fuzzer. Through this solution, this work has three groundbreaking functions: optimized support for the latest Android version 13, automatic extraction of complete service interfaces, and automated testing and monitoring log synchronization.

We collected a total of 118 Android Native system services, including 41 real-name services and 77 anonymous services, and also extracted interface models of 1313 transactions. According to the transaction interface model, the fuzzing test was carried out on the real machine Google Piexl 4xl of Android 13 version, and a large number of crash logs were generated. Through log analysis, we selected 4 typical new vulnerability cases from the newly discovered 13 vulnerabilities for analysis. The experimental results prove the effectiveness of the Fuzzer we developed under the Android 13 version, and propose the feasibility of testing in higher versions in the future.

Keywords: Android Native System Services; Fuzz Testing; Transaction; Dependency; Fuzzer

第一章 作品概述

1.1 研究背景

随着移动互联网的快速发展，智能移动终端逐渐普及，目前最受欢迎的移动操作系统是 Google 公司发布的 Android 系统。调研机构 Counterpoint 近日发布了一份手机操作系统市场调查报告显示，2023 年第一季度，在中国市场，Android 系统占据 72%市场份额，iOS 为 20%，华为鸿蒙系统为 8%，Android 系统仍处在主导地位。**基于 Android 平台的应用程序数目飞速增长，应用程序功能也日益丰富，目前 Android 毫无疑问是全球第一大移动终端操作平台**[3]**。**

Android 平台最大的优势是其开源和开放性，允许任何厂商、用户和移动应用开发者加入到 Android 联盟中，为 Android 打造了极具优势的生态圈。同时带来的问题是 Android 市场碎片化严重，各种版本和系统参差不齐，不同厂商修复补丁、更新系统的频率也不同。360 互联网安全中心最新发布的《2022 年度中国手机安全状况报告》中的系统安全分析结果显示[10]：有 99.9%的用户设备存在至少一个安全漏洞，94.5%的设备存在 5 个以上的安全漏洞，68.8%的设备存在 10 个以上的漏洞，更有 50.4%的设备存在 20 个以上的安全漏洞。根据 Google 近些年发布的安全报告[8]，报告了数百个与 Android 系统服务相关的漏洞，表明 Android 系统服务仍然容易受到攻击，并且对攻击者具有吸引力。

Android 系统服务是为用户应用程序提供基本支持和核心功能的后台运行程序或系统进程。它们通常具有较高的执行权限，提供各种功能和服务，如媒体播放、相机操作等。其中，一些 Native 系统服务以守护进程的形式在系统中运行，例如 statsd 进程。另外，Android 系统服务还可以根据应用领域进行分类，包括普通领域、供应商领域和硬件领域。普通领域的系统服务为普通应用程序提供通用功能和服务，供应商领域的系统服务由设备制造商提供，为特定设备或定制应用提供定制化功能和服务，而硬件领域的系统服务与设备硬件相关，用于控制和管理设备硬件功能。

由于 Android Native 系统服务中的代码是使用 C++语言编写的，它们存在着许多危害严重的安全漏洞。这些漏洞可能被攻击者远程利用，对 Android 系统

进行攻击。**因此，对 Android Native 系统服务进行严格有效的安全测试是非常必要的。**安全测试可以帮助发现和修复 Native 系统服务中的漏洞，防止攻击者利用这些漏洞进行恶意操作。安全测试的目标是识别潜在的安全问题，并提供解决方案以增强系统的安全性。这包括对代码进行审查、静态和动态分析、渗透测试等手段，以验证系统的安全性并修复可能存在的漏洞。**通过对 Android Native 系统服务进行严格有效的安全测试，可以帮助提高系统的整体安全性，减少被攻击的风险，并保护用户的隐私和数据安全。这是保障 Android 系统稳定和可靠运行的重要措施之一。**

表 1-1 Android 系统各版本份额数据对比表

Android 系统版本	2023 年 4 月	2023 年 6 月
Android 13	12.1%	15%
Android 12	16.5%	16.3%
Android 11	23.5%	23.1%
Android 10	18.5%	17.8%
Android 9	12.3%	11.9%
Android 8	6.7%	8.3%
Android 7	3.3%	3%
Android 6	2.5%	2.3%
Android 5	1.9%	1.8%
Android 4	0.6%	0.5%

近日，Google 在 Android Studio 中更新了 2023 年 6 月份的 Android 各版本占比的统计数据[21]，如图 1-1 所示。Android 13（T）目前在全球所有 Android 设备中大约占有 15%的份额。并且根据表 1-1 的数据对比分析，**Android 13 的占比份额已经呈现较快的增长趋势，考虑到此增长趋势，且作为 Android 系统的最新版本，预计其在全球市场的应用也会更加广泛，因此对 Android 13 版本的安全研究具有无可争议的重要性。**

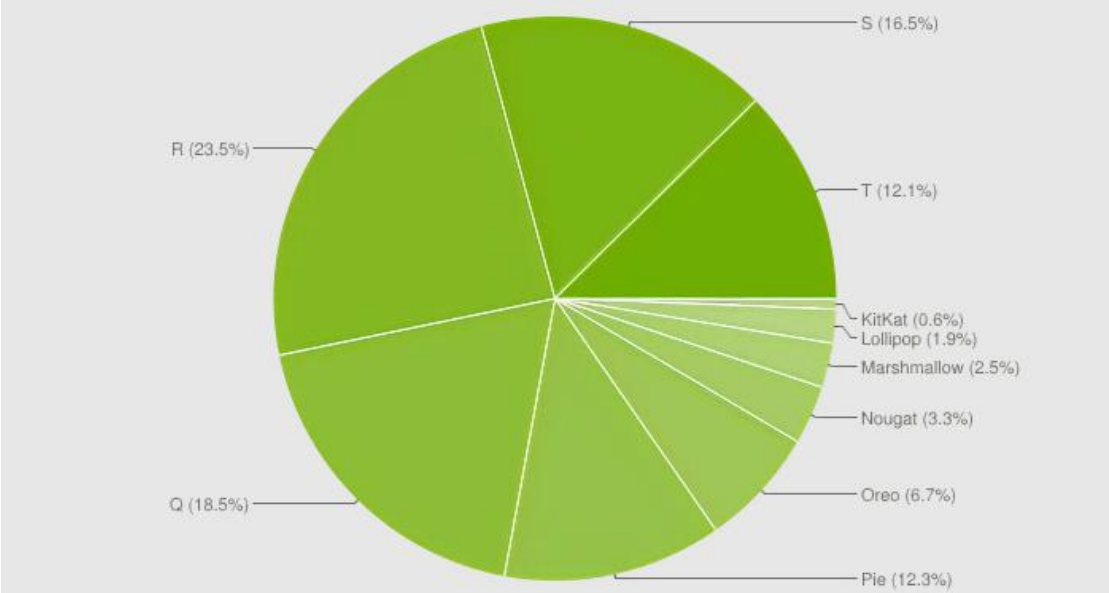


图 1- 1 Google 更新 Android 各版本份额

1.1.1 Android 系统服务介绍

Android 操作系统主要分为应用程序层(Apps)、应用框架层(Java Framework)、Native 层(C++ Framework)和内核层(Kernel)，如图 1-2 所示。应用程序层(Apps)主要负责运行各种应用程序。应用框架层(Java Framework)位于应用程序层(Apps)和 Native 层(C++ Framework)之间，提供应用程序开发所需的框架和程序编程接口（Application Programming Interface, API)。而 Native 层也称为底层库层，包括一些核心的 C/C++库和组件，为应用程序层和应用框架层提供底层的支持和功能。Native 层包括操作系统的核心组件、设备驱动程序和各种系统服务等。内核层负责处理系统资源管理、进程调度、内存管理和设备驱动等底层操作。

在 Android 系统中，安卓系统服务（Android System Service）是位于应用程序层和 Native 层之间的一层抽象，**它们将 Android 提供的各种功能如音频播放（AudioFlinger）、媒体播放（MediaPlayerService)等进行封装，并以应用程序编程接口的形式为上层的应用程序提供服务，所以用户程序仅仅调用公开的接口即可使用硬件功能，无需了解实现细节。**具体而言，根据编程语言的不同，Android 系统服务可以分为两类：（1）Java 系统服务：这些服务主要使用 Java

编程语言实现，并在 Android 框架层运行。常见的 Java 系统服务包括 Activity Manager、WindowManager 和 PowerManager 等。（2）原生系统服务：这些服务主要使用 C++编程语言实现，并在 Android 硬件抽象层(HAL)运行。原生系统服务包括摄像头服务、音频服务和传感器服务等。

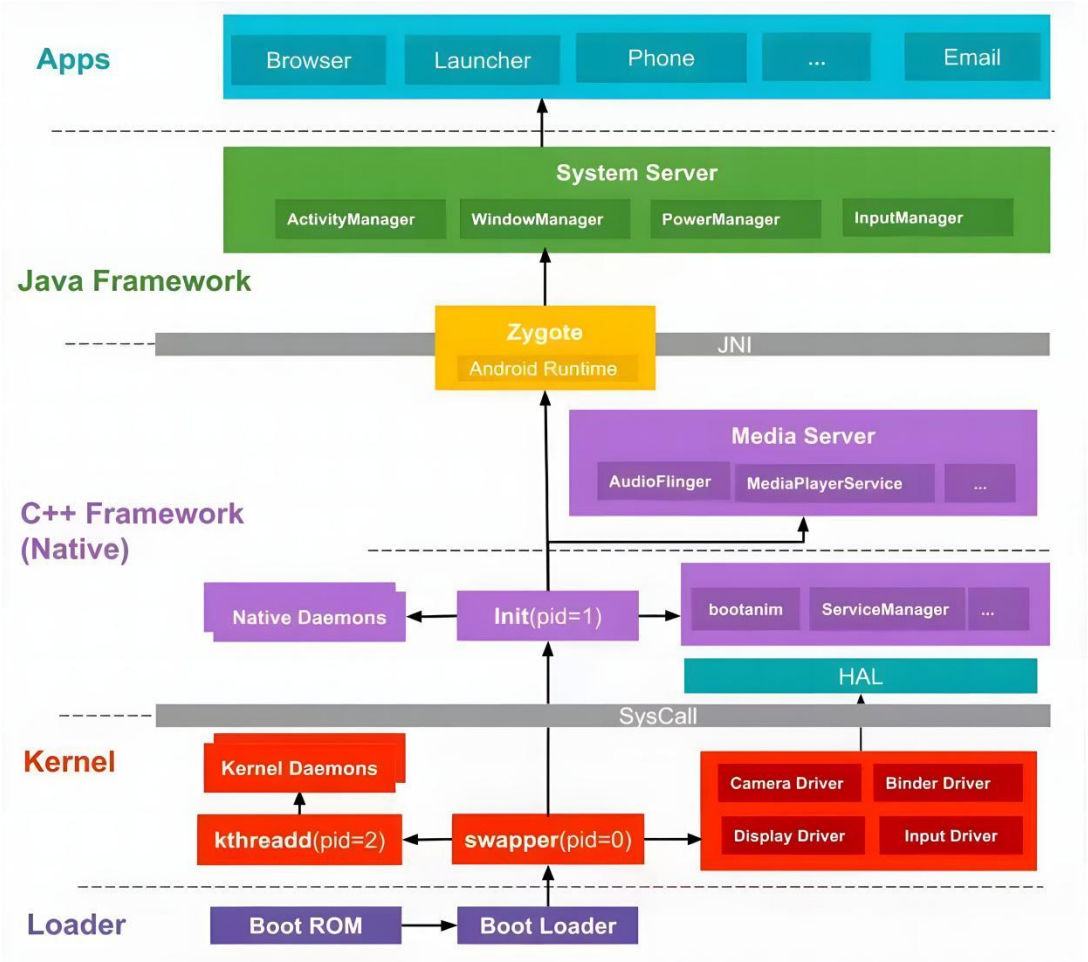


图 1- 2 Google 提供的 Android 系统架构图

本作品所研究的安卓原生系统服务是在 Native 层的基础上构建的一组高级服务，**其通过调用 Native 层的 API 和功能在 Native 层的 HAL 部分（硬件抽象层）运行来实现各种系统级别的操作，同时还提供了高级的应用程序接口供上层应用开发使用。原生系统服务作为应用程序和 Native 层之间的桥梁，承担了将高级功能转化为底层操作的重要角色。**

Android Native 系统服务的安全重要性不言而喻。**Native 系统服务是 Android 系统的核心组件，涉及底层的系统功能和资源管理。由于其特殊的权限**

和系统级别的操作，存在漏洞可能导致系统安全受到威胁，如数据泄露、远程攻击等。此外，Native 系统服务作为 Android 系统的基础组件，被广泛使用于各种应用程序和设备中。一旦存在漏洞，可能影响大量的用户和设备。加强对 Android Native 系统服务的安全研究，及时发现其中可能的安全漏洞，对于保障 Android 系统的安全至关重要。

1.2 研究现状

1.2.1 安卓系统安全

安卓系统安全是指保护安卓操作系统及其相关组件、应用程序和用户数据免受各种威胁和攻击的综合性措施和技术。**安卓系统安全涉及多个方面，包括应用程序安全、网络安全、数据安全以及安卓原生系统服务安全[4]。**安卓系统安全的重要性日益凸显，因为安卓设备在日常生活中扮演着重要的角色，包括个人通信、移动支付、社交媒体、在线购物等。

安卓系统应用程序安全直接影响到安卓用户数据隐私安全和金融财产安全，这一方面的研究侧重于安全漏洞挖掘与修复与权限隐私保护。前者的研究主要是通过模糊测试等一系列漏洞挖掘与检测技术来分析。他们关注常见的漏洞类型，如输入验证错误、缓冲区溢出、权限绕过等。2014 年，Steven Arzt 等人在论文"DroidBench: A Benchmark Suite for Android App Analysis"提出了 DroidBench 基准测试套件，用于评估和比较安卓应用程序分析工具的性能和准确性。同时，近年还有许多工具也随研究诞生，如 QARK (Quick Android Review Kit)：一款开源的安卓应用程序漏洞扫描工具，用于检测应用程序中的安全漏洞和潜在风险。

安卓系统网络安全涉及到保护安卓设备在网络环境中的安全性和防御各种网络威胁，这方面的研究侧重于恶意应用程序及软件和网络攻击防御。其研究与安卓系统的应用程序安全总是相关联的。2010 年，W. Enck 等人创造了 TaintDroid 工具，用于实时监控和跟踪安卓应用程序中的信息流和隐私泄露；2012 年，K. R. Kandula 等人在研究中提出了一种通过追踪应用程序清单文件和 API 调用来检测安卓恶意软件的方法。

安卓原生系统服务安全中，原生服务系统安全研究的侧重点包括漏洞分析与修复、权限管理与访问控制、安全加固和保护、安全审计和监控等。2013 年，Y.Zhou 等人在论文“Understanding Android Binder: A Security Perspective”中探讨了 Android Binder 的安全问题和漏洞分析方法。2014 年，Y. Zhang 等人在论文“Moat: Sensitive Data Leakage Prevention for Mobile Devices”提出了 Moat，一种保护 Android 系统服务中敏感数据泄露的解决方案。

我们调研了近五年 SCI 收录的 Android 系统安全相关的论文，通过比较分析发现，其中大约 47%的论文有关应用程序安全，21%的论文与网络安全有关，32%的论文与数据安全有关，**其中不乏有些论文涉及了多个方面。但是，详细关于安卓原生系统服务安全及其模糊测的相关论文少之又少，体现了对此方面的研究还处于起步阶段，且缺乏研究。**其原因也在于应用程序、网络安全、数据安全这些方面应用更广泛。其比例如图 1-3 所示。

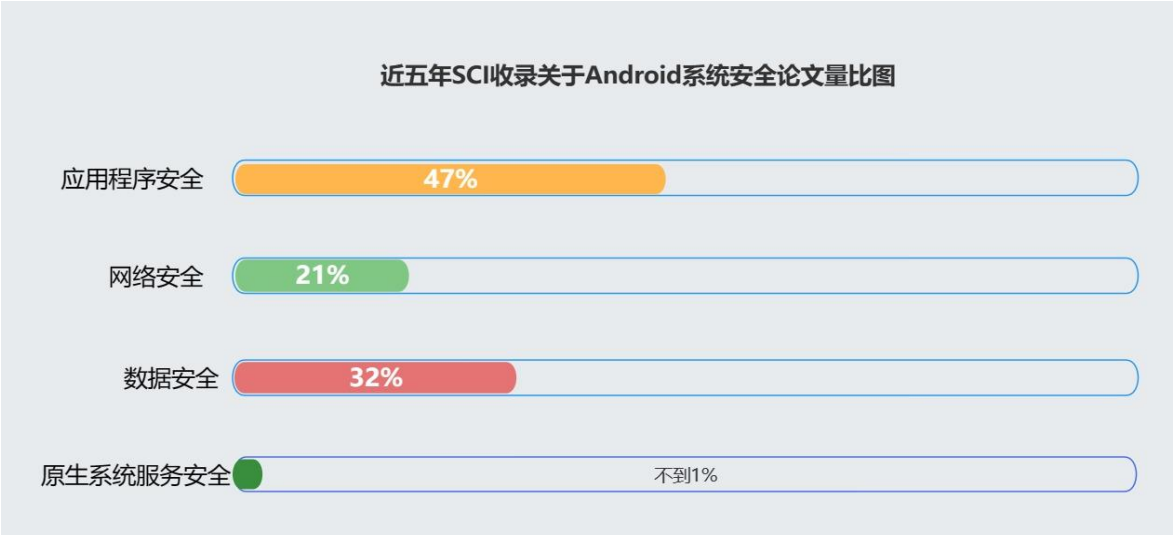


图 1- 3 近五年 SCI 收录关于 Android 系统安全论文量比图

1.2.2 模糊测试与安卓系统服务

随着时间的推移，模糊测试逐渐被应用于各个软件领域，包括操作系统、编译器、浏览器、应用程序等。模糊测试成为发现软件漏洞和错误的一种有效方法，也为改善软件的鲁棒性和安全性做出了重要贡献。

在当前众多成熟的模糊测试工具中，American Fuzzy Lop (AFL) 是目前广泛应用且备受认可的模糊测试工具之一。AFL 采用了指令引导的遗传模糊器方法，

在目标程序的编译阶段进行插桩，以便在模糊测试期间探测目标程序内部的执行路径。AFL 的模糊输入构造基于代码块覆盖引导，通过对已有输入的变异来改善模糊测试的质量。此外，AFL 还支持无源码的运行时插桩，并可借助 QEMU 虚拟机跟踪目标程序的执行指令，尽管这会带来一些运行时开销和仿真环境的依赖。另外一个基于覆盖引导的模糊器例子是 syzkaller，它同样使用了 QEMU 和编译期插桩的方式，通过系统调用接口对整个 Linux 内核进行模糊测试。与其他模糊器相比，AFL-Fuzz 具有较低的性能消耗和高效的模糊测试策略，只需要简单的配置即可处理复杂的程序。此外，还有一些基于 AFL 进行改进的衍生模糊器，如 winafl、libfuzzer 等。这些工具的发展和应用为软件安全领域的漏洞发现提供了重要的支持。研究人员和安全专家利用这些工具来提高软件系统的安全性，并不断探索和创新新的模糊测试技术然而，尽管模糊测试已经被应用于 Android 的相关研究中，但据我们了解，现有研究很少针对 Android 原生系统服务进行深入探讨。目前，已经提出了两种自动化模糊测试方法以发现 Android 系统服务中的漏洞，但除此，之外，仅有一种不可扩展的手动方法。

第一个模糊测试方案是 Binder-Cracker[7],其通过记录 30 个热门应用程序发出的请求来捕获目标服务的输入模型。这种方法存在一个固有缺陷，即它无法精确地重新覆盖输入语义，例如变量名称和类型。此外，由于测试覆盖范围不全，它将无法涉及那些很少使用或深度嵌套的接口。不仅，BinderCracker 框架还存在一些限制。首先，该框架中的一些模块需要修改并重新编译 Android 底层代码，因此不适用于厂商定制的 Android 系统。这限制了框架的适用范围。其次，在前期数据收集环节，BinderCracker 仅提供了方法论，未给出清晰且先进的数据收集方案，这可能导致在模糊测试的过程中数据收集的不完整或不准确。

第二个模糊测试方案是 Cao 等人提出的 Buzzer[6],主要关注 Android 系统服务的输入验证漏洞，它旨在通过自动化生成和变异输入数据来探测软件系统中的潜在漏洞。它从 Web 应用程序中的问题解决方案中进行了知识迁移。然而，Buzzer 方案存在一些缺点，其中之一是在前期数据收集环节过程中依赖大量的人工操作。例如，需要通过人工审计源码的方式来筛选具有输入验证特征的服务方法。

还有一个模糊测试方案是Chizpurflel[11],它利用Java反射机制获取接口的参数类型，以测试供应商实现的Java服务。然而，这种方法并不适用于检索Android原生系统服务(Native)的输入模型。

换而言之，尽管现有研究尝试利用模糊测试方法发现Android系统服务中的漏洞，但这些方法要么无法精确覆盖输入语义，要么无法应用于Android原生系统服务(Native)。Binder-Cracker方法受限于其测试覆盖范围，而Chizpurfle方法则仅适用于Java服务，而非原生系统服务(Native)。

因此，综合来看针对Android原生系统服务，现有研究尚未提出完全适配的模糊测试方案。本作品的研究颇具价值。

1.3 作品主要工作

本作品旨在实现对Android Native系统服务的模糊测试，实现的模糊测试器总体架构如图1-4所示，其详细介绍如下：

(1)服务接口文件收集：通过对Android Open Source Project(AOSP)源码进行编译并记录编译命令，通过读取编译命令收集所有与Android Native系统服务相关的文件，包括C++编写的系统服务和Android Interface Definition Language(AIDL)动态生成的系统服务；

(2)接口模型预处理：根据Clang Abstract Syntax Tree(Clang AST)的特性，利用插件提取树形结构，并以XML文件形式保存；

(3)接口模型后处理：通过保存了onTransact函数AST特性的xml文件，对各个节点进行处理，例如ifStmt,ForStmt等，处理变量之间的约束和依赖，并保存在JSON格式的文件中；

(4)依赖推断：通过提取的接口模型，推断出事务间依赖、服务接口之间的依赖，并生成接口依赖图；

(5)Fuzzer设计与实现：完成上述工作之后，开始对Fuzzer进行设计和实现，Fuzzer主要工作需要根据接口模型和依赖关系生成测试用例，在Fuzzer中，事务生成器主要构造具体事务需要的输入变量，并处理约束和依赖关系；执行器主要获取具体待测的服务接口，并调用onTransact输入测试用例；

(6)Manager设计与实现：为了让Fuzzer自动运行并监控设备状态，本文实

现了 Manager 模块来管理 Fuzzer，例如，确保设备状态、上传接口模型和 Fuzzer、同步日志等功能；

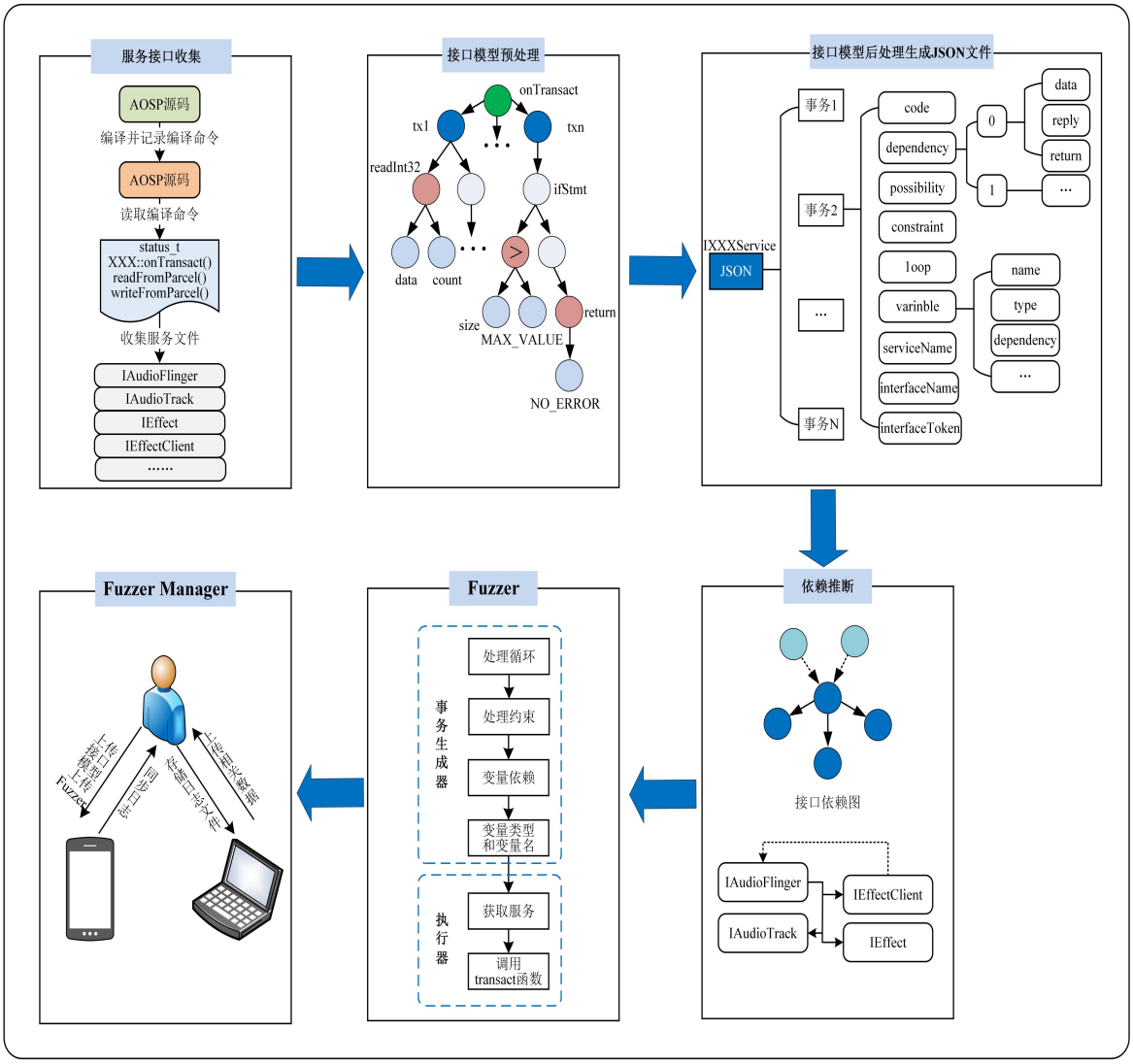


图 1-4 作品总体框架

1.4 特色描述

1.4.1 漏洞挖掘覆盖率全面

在我们的研究中，我们仔细分析了 Android 原生系统服务之间的接口关系，包括服务之间的直接依赖关系以及多级接口之间的间接依赖关系。我们通过审查

源代码、静态分析和动态调试等方法，深入探索了这些依赖关系的细节，并进行了详尽的文档和记录。

通过挖掘多级接口，我们发现了在传统分析中可能被忽略的深层次的依赖关系。这些多级接口涉及到不同层级的系统服务之间的交互，对于理解系统的整体架构和功能至关重要。**我们对这些多级接口进行了全面的调研和分析，包括其功能、输入输出约束、安全性和潜在的漏洞。**

我们的研究成果为 Android 原生系统服务的安全评估和漏洞发现提供了权威和专业的基础。通过深入研究接口之间的依赖关系和多级接口，我们能够更准确地评估系统的安全性，并发现潜在的漏洞和安全隐患。**这些研究成果可以为安卓系统开发者、安全研究人员提供有价值的参考和指导，以促进安卓系统的安全性和可靠性进一步提升。**

1.4.2 自动化提取与检测

以往部分类似工具不足以满足对 Android 原生系统服务进行全方面全流程地自动化检测，我们基于 Android 原生系统服务的接口特点、依赖关系等特征，设计了 interface collector（接口收集器）、 interface model extractor（接口模型提取器）、 dependency inferer（依赖关系推断器）和 fuzzer engine（模糊器引擎）。**例如，针对接口模型提取器，本作品提出了一种从 AST 中自动提取输入接口模型和语义的解决方案，这种方法可以应用于其他基于接口的程序。**并且对接口收集、关系推断等实操步骤都是自动化进行。以往传统的自动化工具对漏洞分析不当或者不全面，可能会导致一些潜在的漏洞未被发现，而本作品的工具针对 Android 特征特色设计，对漏洞的挖掘程度更深、范围更全。同时，通过模糊测试的自动化过程，我们可以快速定位可能的系统漏洞和故障。**这大大减少了手动测试和调试的时间和成本。**

1.4.3 多版本适用性和易调整性

本作品是针对 Android 13 版本进行测试。考虑到不同版本的原生系统服务可能存在不同的漏洞和安全问题，比如在 Android 12 和 Android 13 之间的差异

可能导致一些新的安全问题，因此我们在设计工具时，特别注重对未来不断更新的 Android 版本的适用性和易调整性。我们的工具考虑了 AOSP（Android Open Source Project）的源码特性、Binder IPC 通信机制的核心重要性，以及语义信息的相似性等因素。因此，我们的服务接口文件收集，获取并生成服务接口模型，以及之后的推断依赖关系等环节具有多版本实用性。例如，我们可以通过自动化工具快速提取和更新不同版本系统的服务接口信息。**当我们需要对其他 Android 版本进行 Native 系统服务的模糊测试时，我们只需下载并导入对应的 Android 系统版本，以及微调所变动的用户自定义服务接口即可。故可以有效地适应 Android 系统的快速迭代，因此本作品还具有较强的易调整性**。

1.5 应用前景

1.5.1 适应多厂商环境和商业市场广阔

Android 系统最大优势是开源性，允许任何移动终端厂商、用户和应用开发商加入到 Android 联盟中来，允许众多的厂商推出功能各具特色的应用产品，导致系统服务的实现方式存在着差异。在安卓系统的定制和修改过程中，厂商通常会对一些接口和 API 进行调整和扩展以及系统设置和选项的调整和功能增强和优化等，以适应其定制化的功能和硬件需求。**针对不同的厂商，基于本作品的基础，开发者需要根据不同厂商具体改变而修改对应检测代码，即可对各具特色的应用产品进行初步地检测漏洞。这种适应性对于商业市场来说至关重要，因为它可以为各个厂商提供定制化的模糊测试解决方案，满足他们对系统服务的安全性和稳定性的要求,有助于厂商提升其产品的质量和可靠性。同时，根据测试结果快速迭代和优化产品，有利于厂商在竞争激烈的移动设备市场中保持竞争优势。综上，本作品的衍生与拓展极具商业价值。**

1.5.2 安全合规性自动化验证平台

本作品具有与基于不同指标的模糊测试工具及其他自动化安全测试工具集成的能力，能够建立一个全面的安全合规性自动化验证平台。

该平台根据不同的安全标准和法规要求，自动运行模糊测试和其他安全测试，快速验证系统服务的安全性和合规性，并生成详细的安全合规性报告。

在该平台中，不同类型的模糊测试工具可以相互配合，通过组合运行，提高测试的全面性和覆盖范围。**例如，基于生成的模糊测试用于发现目标系统的新漏洞，基于变异的模糊测试扩展测试样本集，增加系统服务的覆盖率，而基于覆盖率的测试着重测试代码路径覆盖情况，进一步检测可能存在的安全隐患。**

同时，该平台还集成其他自动化安全测试工具，如静态代码分析工具、漏洞扫描工具、安全配置检查工具等，用于对系统服务进行全方位的安全审查和评估。通过多个安全测试工具的联合使用，可以从不同角度发现和修复安全漏洞，确保系统服务在设计、开发和部署过程中的安全性和合规性。

在实际应用中，**用户可以根据特定的安全标准和法规要求配置测试方案**，并选择合适的测试工具进行集成和运行。平台会自动执行测试任务，收集测试结果，并生成详尽的测试报告，展示系统服务的安全性和合规性情况。这样的自动化验证平台将极大地提高安全合规性验证的效率和准确性，为用户提供全面的安全保障和合规性验证支持。

总的来说，该安全合规性自动化验证平台的全面性和灵活性使其成为企业、开发者和安全团队的得力助手，为他们提供快速、准确、可靠的安全测试和合规性验证服务，帮助他们构建更加安全、可信赖的系统服务。**这样的创新性平台在安全领域具有广阔的应用前景，并能够满足不同行业和领域的安全合规性需求。**

1.5.3 Android ROM 定制

本作品自动提取 Native 系统服务接口模型，为 Android ROM 定制提供必备参考，以更易理解和控制系统行为。开发者可通过改变系统外观、用户界面、应用和核心功能，满足特定需求，打造个性化体验，提供消费者多样化选择。接口模型信息让开发者精准理解系统服务工作原理，找出并优化性能瓶颈，提升效率，也可针对硬件配置优化 ROM，延长电池寿命。

接口模型助力开发者创新定制系统服务，增强设备安全性。ROM 定制能为设备提供原厂 ROM 未覆盖的功能，满足企业级需求，如加强数据保护，远程管理，集成内部应用等。基于 Android 开源特性，ROM 定制是广阔且活跃的领域。**通过 Native 系统服务接口模型的自动提取，本作品为 ROM 定制领域带来新的发展动力，可能推动企业级移动设备的发展，具有广阔的应用前景。**

1.6 小结

本章节首先基于 Android 系统的全球市场和各个领域的广泛应用，强调了 Android 系统安全的重要性，分析了其面临的严峻安全现状。随后介绍了 Android 原生系统服务的概念、功能以及安全重要性，但是发现目前关于 Android 原生系统服务安全的研究仍然相对不足，并且在 Android 系统服务的模糊测试方面，虽然目前已经提出一些针对 Android 系统服务的模糊测试方案，如 Binder-Cracker 和 Buzzer，但它们存在测试覆盖范围不全、人工操作依赖等问题，并不能完全适用于 Android 原生系统服务。设计本作品即是为了填补不能完整、全面地对 Android 原生系统服务进行模糊测试这个空白。随后介绍本作品通过服务接口文件收集、接口模型预处理、接口模型后处理、依赖推断、Fuzzer 设计与实现、Manager 设计与实现六个主要工作，提供了有望为安卓原生系统服务安全提供全面的模糊测试解决方案。本作品具备全面的漏洞挖掘和自动化检测能力，适应多版本和多厂商环境。应用前景广阔，可提升系统服务的安全性和稳定性，并满足持续集成和批量测试的需求。

第二章 作品设计及实现

2.1 相关技术及理论

2.1.1 模糊测试技术(Fuzzing)

模糊测试（Fuzzing）[2]是一基于黑盒的安全测试技术，它广泛应用于软件安全领域，旨在发现软件系统中的漏洞和错误。模糊测试通过向目标软件系统注入大量随机、异常或无效的输入数据，观察系统的响应情况，并检测是否存在异常行为或崩溃等情况。模糊测试的基本原理是利用不正确或非预期的输入来揭示潜在的软件漏洞，以便开发人员可以修复它们。

模糊测试技术的核心在于生成测试用例。良好的测试用例生成技术可以确保更高的代码覆盖率和测试效率。测试用例生成技术可以分为两类：基于生成和基于变异。

基于生成的测试用例生成技术是根据事先定义好的模型或规则，可以生成大量符合规定的测试用例。这种方法可以帮助覆盖系统的不同路径和边界条件，从而发现潜在的漏洞或错误。

基于变异的测试用例生成技术是根据已有的测试用例进行不断的变异生成新的测试用例。通过对现有测试用例进行变异，例如改变输入数据、修改参数值等，可以生成一系列略有差异的测试用例。这种方法能够探索系统的不同执行路径和输入组合，以发现可能存在的问题。

在 Android 原生系统服务中，模糊测试被广泛应用于漏洞检测和安全性评估。通过模糊测试，可以探测并揭示系统中的安全漏洞，例如缓冲区溢出、输入验证错误、访问控制问题等。模糊测试可以在系统中对服务接口自动化地注入大量随机、异常或无效的输入，以验证系统对这些输入的处理是否安全可靠。**本作品采用的是基于生成的 Fuzzing 测试方法，它通过提取事务接口模型来生成大量的测试用例进行 Fuzzing 测试。**通过事务接口模型的提取，可以根据模型生成大量具有多样性的测试用例，以便广泛地测试系统的各个方面和功能。

Fuzzing 测试的工作流程一般如下图 2-1 所示：

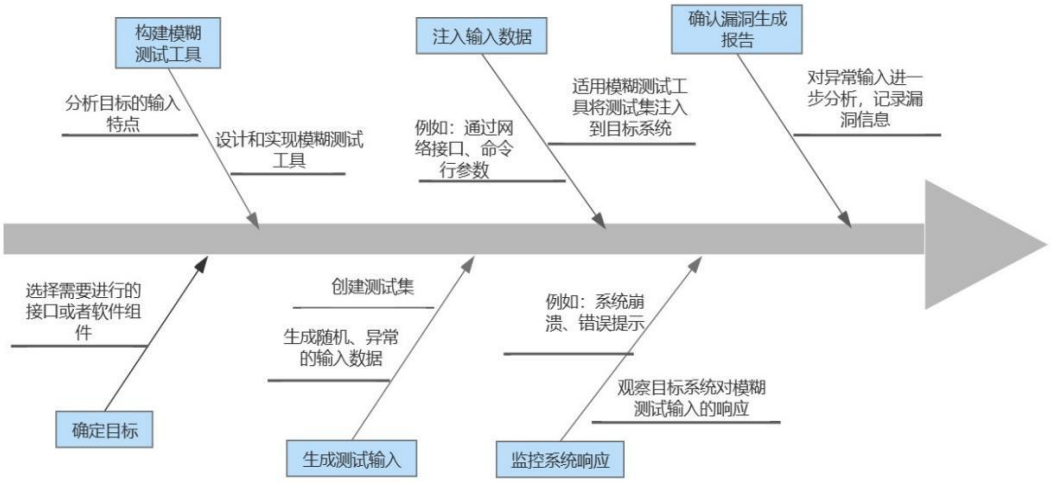


图 2- 1 模糊测试流程图

2.1.2 Binder 通信机制

为了保护应用和系统免受恶意应用的攻击，Android 采用了基于用户的 Linux 保护机制，为每个 Android 应用设置一个内核级应用沙盒。默认情况下，应用不能彼此交互，而且对操作系统的访问权限会受到限制。在应用间需要交互的情况下，需要使用进程间通信方式（IPC）。与 Linux 不同的是，Android 系统服务采用 Binder 通信机制[16]进行进程间通信（IPC）。**Binder 是一种高效、轻量级的通信框架，通过代理和存根对象实现客户端和服务端之间的通信。**当一个应用程序需要与系统服务进行通信时，它会通过服务管理器（ServiceManager）获取服务的代理（Proxy）对象，然后通过该代理对象调用服务的接口。在服务端，存根（Stub）对象负责处理来自代理对象的请求，执行相应的操作，并将结果返回给代理对象。

Andriod 系统的 Binder 通信机制模型主要由四部分构成[18]，即 Cilent(客户端)、Server(服务端)、Service Manager(服务管理器)、Binder Driver(Binder 驱动,一般指路由器），如下图 2-2 所示。

在 Android 系统中，Client、Server 和 Service Manager 运行在用户空间，而 Binder Driver 运行在内核空间。Binder Driver 通过设备文件 dev/binder 提供接口，用户空间可以使用 open、ioctl、mmap 等方法与其进行交互。Service Manager 的主要作用是管理系统中的服务。Server 通过 addService()方法将自

已注册到 Service Manager，而 Client 通过 getService()方法从 Service Manager 获取所需的 Server 信息。Client 和 Server 之间的通信是通过 Proxy-Stub 设计模式实现的。在这个模式中，AIDL（Android Interface Definition Language）工具会生成代理 Proxy 和存根 Stub 对象。Client 通过代理 Proxy 调用 transact()方法将待传输的数据打包成 Parcel，并发送给 Server。而 Server 则通过存根 Stub 的 onTransact()方法接收 Parcel 数据，并进行解析和处理。整个 Binder 通信的实现是通过底层的 Binder 驱动程序间接完成的。Binder 驱动负责处理数据的传输、跨进程调用和进程间通信的维护。它提供了高效的线程间通信机制，确保数据的安全性和可靠性。

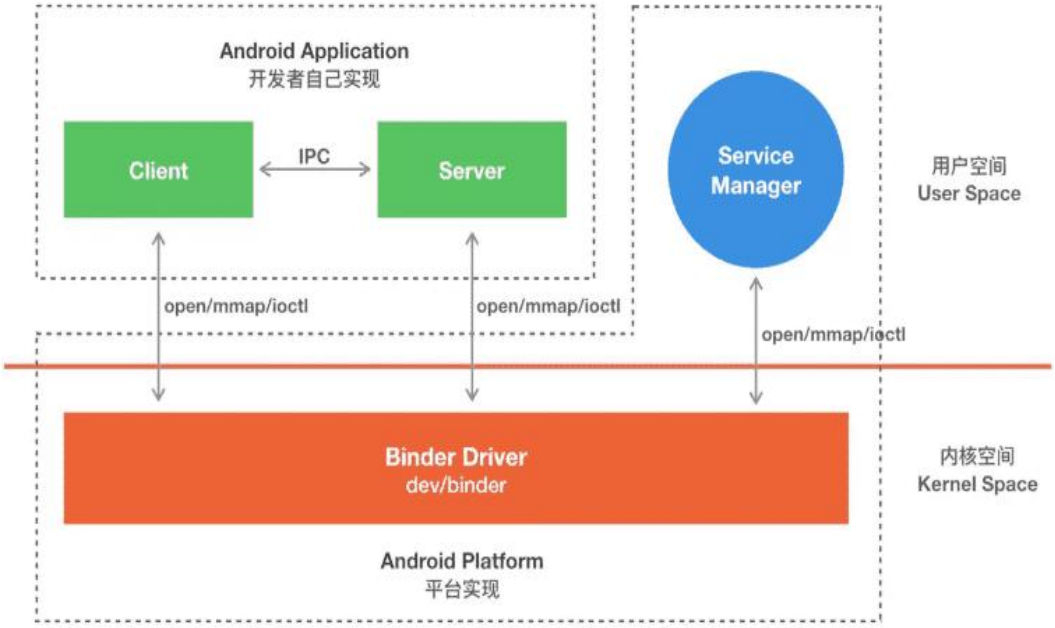


图 2- 2 Binder 通信机制架构

2.1.3 Native 系统服务接口调用与 Android 接口定义语言(AIDL)

2.1.3.1 Native 服务接口调用

客户端能够调用本地系统服务接口，并与服务端进行通信和交互。这种流程可以实现客户端和服务端之间的远程通信，使客户端能够访问系统提供的各种功能和服务。其流程主要如下图 2-3 所示：

（1） **系统启动时注册服务**：在系统启动时，本地服务以"名字.引用"的形式自动注册到 ServiceManager 中。这些注册到 ServiceManager 的服务接口被称为实名服务；

（2） **客户端发起服务请求**：当客户端需要调用系统服务时，它会向 ServiceManager 发起服务请求。客户端指定所需服务的名称或标识；

（3） **ServiceManager 返回服务代理**：ServiceManager 接收到服务请求后，会返回该系统服务接口的代理给客户端。服务代理是客户端与服务端通信的桥梁；

（4） **远程过程调用（RPC）**：客户端利用服务代理通过统一的 transact 函数进行远程过程调用。客户端将请求传递给服务代理，代理负责将请求发送给服务端；

（5） **服务端处理请求**：服务端接收到请求后，会通过 onTransact 函数进行处理。根据请求的类型，分为两种情况；

a. 调用具体的事务：如果请求是调用具体的事务，代理接口会直接调用服务端的具体服务函数，并将执行结果返回给客户端。

b. 请求匿名服务接口：如果请求是请求匿名服务接口，客户端通过代理接口向服务端发起请求，并返回的是代理匿名服务接口。

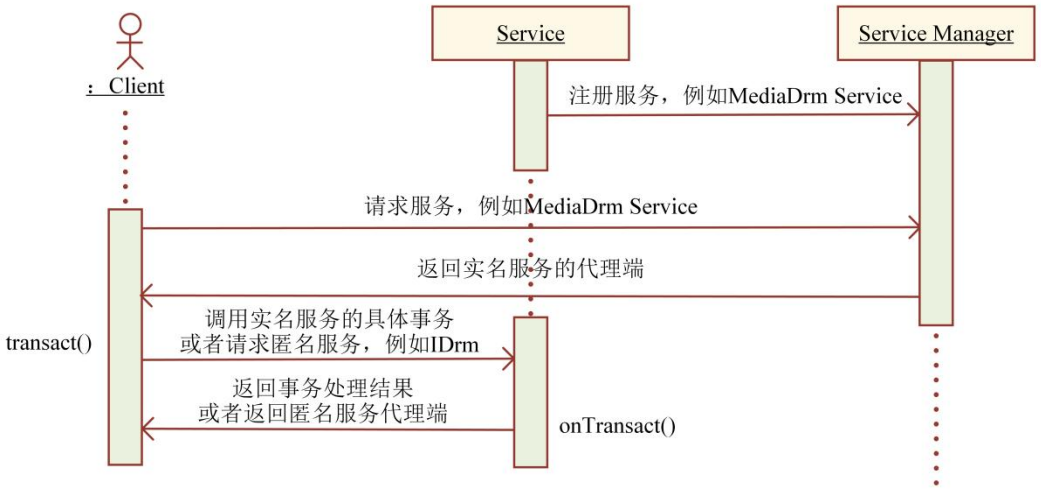


图 2- 3 Native 系统服务调用程序

在 Android 中，Client(客户端）通过查询 ServiceManager 来获取目标服务的代理接口。然后，客户端使用 Bp:transact(code, data, reply, flags)函数

返回实名服务的代理端。Service(服务端)通过 BBinder:onTransact(code, data, reply, flags)函数来处理客户端的调用实名服务具体事务的请求，并根据事务的代码将请求转发到特定的事务处理代码段，然后返回处理结果。

事务代码 code 确定执行目标服务的具体操作。参数 data 是一个序列化了各种数据类型的 Parcel 对象，用于传递请求数据给服务端。服务端处理完请求后，将结果序列化到 Parcel 类的 reply 对象中。参数 flags 的值决定调用过程是同步还是异步的。0 表示同步调用，客户端等待系统服务返回数据，而 1 表示异步调用，客户端不等待系统服务返回数据。在模糊测试中，为了提高效率，Fuzzer 默认采用异步调用方式（flags 为 1）。当 Fuzzer 需要从 reply 中获取返回数据时，将 flags 设置为 0 以获得返回数据。

为了对所有的接口事务进行模糊测试，本作品选择在 Server 端的 BBinder:onTransact()函数中提取所有事务的接口模型，主要有以下原因：

（1） 方便扫描和提取模型：在 Client 端，同一个服务的不同事务可能存在于不同的源文件中，而在 Server 端，所有事务会被统一处理在一个文件中，使用 switch-case 语句进行分发。这样做可以避免对每个源文件进行扫描和提取模型的复杂性。

（2） 完整的输入输出信息：在 Client 端，请求数据会使用 writeXXX()函数序列化到 Parcel 类的 data 对象中,而在 Server 端会使用对应的 readXXX()函数将 data 中的输入数据进行反序列化。因此，在接口模型处理中可以提取到完整的输入信息。类似地，对于输出信息，Server 端会使用 writeXXX()函数将输出数据序列化到 Parcel 类的 reply 对象中。

（3） 适用于实名和匿名服务：该通信流程不仅适用于注册在 ServiceManager 中的实名服务，对于匿名服务也采用了相同的通信结构。因此，本文能够提取匿名服务的接口模型。

通过以上的方式，可以对所有的接口事务进行模糊测试，并获取到完整的输入输出信息，从而进行有效的测试。

Android 系统服务接口存在着权限绕过、拒绝服务、远程代码执行以及跨越界限攻击等漏洞，这些漏洞和危害对 Android 系统的安全性构成潜在威胁，可能导致用户隐私泄露、设备受损、系统不稳定等问题。因此，科研工作者和开发人

员需要重视系统服务接口的安全性，进行充分的输入验证、输出过滤和权限控制，同时及时更新和修补系统以纠正已知的漏洞，以保护用户和设备的安全。

2.1.3.2 Android 接口定义语言（AIDL）

AIDL 是一种专门用于 Android 系统服务接口定义的语言。它用于描述系统服务和应用程序之间的通信协议。AIDL 允许开发者定义可供其他进程调用的接口，这些接口可以自动处理跨进程通信和数据序列化/反序列化的细节。**AIDL 提供了一种简单而强大的方式来定义接口，并生成相应的代理和存根代码，以便在不同的进程之间进行数据交换，**具体而言是在构建 Android 系统时，AIDL 文件被编译成相应的 Java 和 C++代码，这些代码随后用于实现客户端代理和服务器端存根。

需要注意的是，Native 服务接口调用通常是在本地进程中进行，而 AIDL 主要用于跨进程通信。如果本地服务接口使用 AIDL 进行定义，那么客户端可以通过 AIDL 生成的代理类来进行跨进程通信，实现与本地服务的交互。但如果本地服务是通过其他方式定义的，如使用 C++编写的 Native 接口，那么 AIDL 与 Native 服务接口调用之间可能并没有直接的关系，而是使用其他方式进行本地服务的调用和交互。

AIDL 也可能存在一些安全漏洞和问题，其中一些主要的漏洞如下图 2-4 所示：

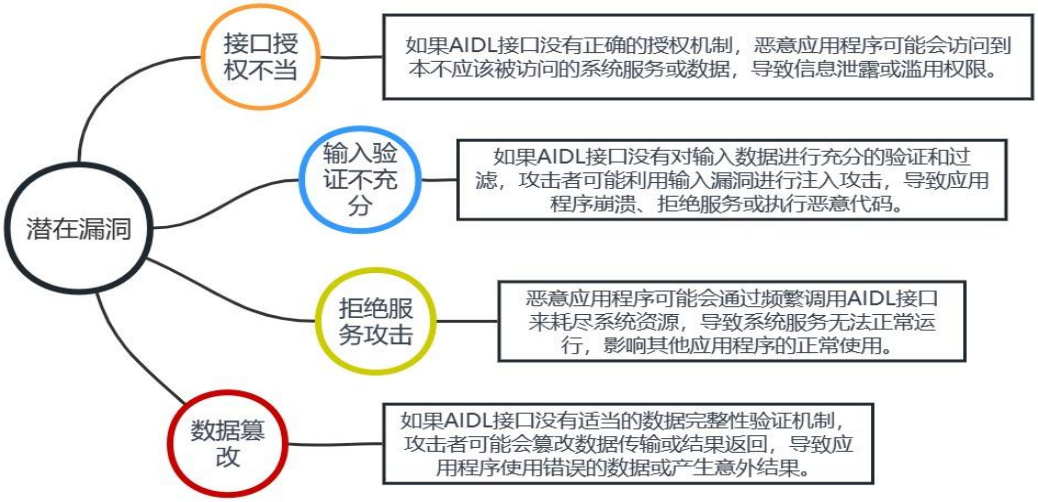


图 2- 4 AIDL 潜在漏洞类型

2.1.4 Android flash

Android 刷机(flash)指的是将设备的固件(也称为 ROM)更换为自定义的固件，以实现各种定制化需求的过程。刷机可以帮助用户解锁设备的潜力，包括增强性能、安全性和功能等。简单来说刷机就是为手机设备刷入系统，刷入的系统可以由第三方厂商定制也可以由开发者修改 AOSP 添加自定义功能后再制入手机设备。

（1）BootLoader

在操作系统内核运行之前 CPU 会先执行一段已经预设好的程序，该程序就是 BootLoader。机器启动之初可以通过这段小程序，进行硬件初始化，获取内存大小信息等，调整手机到适配状态。简言之，可以通过 BootLoader 将 Android 系统的软硬件环境调整到一个适用于操作系统内核调用的运行环境。

（2）FastBoot

fastboot 也叫快速启动模式，是 BootLoader 程序后期的一个特殊阶段。开发者可以通过 USB 数据线与将手机与电脑连接，然后在电脑上执行一些 adb 命令，如刷 Android 系统镜像到手机上。

（3）Recovery

Recovery 模式指的是一种可以对 Android 内部的数据或系统进行修改的模式，也叫恢复模式，可以做一些删除备份、管理的工作。

（4）Android 调试桥(ADB)

ADB 是一种命令行工具，它使开发人员能够与安装了 Android 操作系统的设备进行通信，并对设备进行调试和测试。ADB 通过 USB 连接将设备连接到计算机，并提供了一系列命令，使开发人员能够访问和控制设备上的各种功能和设置。

本文针对的是 Android Native 系统服务，因此需要对指定版本的 AOSP 源码进行编译，并将生成的镜像对 Android 设备进行刷机，这样才能将 Fuzzer 运行在指定 Android 版本的真机设备中；此外还需要通过主机来控制设备，所以需要使用 ADB 命令行工具来执行相关命令，如运行 Fuzzer、推送和拉取文件、日志同步等。

2.1.5 HWAddressSanitizer 工具

HWAddressSanitizer (HWASan)[21] 是一款类似于 AddressSanitizer 的面向 C/C++语言的快速内存错误检测工具，旨在帮助开发人员发现和调试应用程序中的内存错误问题。与 ASan 相比，HWASan 使用的内存少得多，因而更适合用于整个系统的清理，以及 HWASan 仅适用于 Android 10 及更高版本。

HWASan 工具通过在源码编译时对汇编代码进行插桩，运行时对内存访问进行监控和检查。通过编译时插桩和运行时监控，HWASan 能够对应用程序的内存访问进行细粒度的监控和检查。它可以捕获和报告缓冲区溢出、使用已释放内存、内存越界访问等内存错误，帮助开发人员在早期发现和修复这些问题，提高软件的质量和安全性。

HWAsan 可以检查的内存错误主要包括以下：

①Use after free

使用已释放的内存，堆上分配的空间 fiee 之后被再次使用。

②Heap buffer overflow

堆缓冲访问溢出，访问的区域在堆上，且超过了分配的空间。

③Stack buffer overflow

栈缓冲访问溢出，访问的区域在栈上，且超过了分配给它的空间。

④Global buffer overflow

全局缓冲访问溢出，访问的区域是全局变量，且超过了分配给它的空间。

⑤Use after return

Return 后使用，函数在栈上的局部变量在函数返回后被使用，默认不开启。

⑥Use after scope

在作用域外使用(局部变量离开作用域以后继续使用)。

⑦Initialization order bugs

初始化顺序错误，检查全局变量或静态变量初始化的时候有没有利用未初始化的变量默认不开启。

⑧Memory leaks

内存泄漏，未释放堆上分配的内存。

在本作品中，采用 HWASan 来全局编译 AOSP 源码，以此来检查 Android Native 系统服务调用过程中的内存错误问题。同时，我们使用自动生成的符合 Native 系统服务语义的输入来进行模糊测试。**通过不断调用系统服务并触发各种边界情况和异常场景，我们可以生成一系列的测试用例。模糊测试的输出便是产生的崩溃日志文件，也就是本作品的 tombstone 文件**[17]。这些 tombstone 文件是由于某些 C/C++库函数的错误或 HWASan 检测到的内存错误问题导致进程崩溃而生成的。它们提供了关于崩溃的详细信息，包括堆栈跟踪、寄存器状态以及触发崩溃的具体操作。**通过分析这些 tombstone 文件，我们可以识别和定位 Android Native 系统服务中的内存错误问题。**

HWASan 工具在这个过程中扮演着核心的角色。它通过在编译时插入额外的代码和元数据，实时监控内存访问并检测内存错误。当 HWASan 检测到内存错误时，它会触发进程崩溃并生成相应的 tombstone 文件。HWASan 的检测能力帮助我们及时发现并定位潜在的内存错误，为分析和修复提供了重要的线索和信息。**因此，通过使用自动生成的输入和 HWASan 工具的支持，本文能够有效地发现 Android Native 系统服务中的内存错误问题，并生成相应的 tombstone 文件以供进一步分析和调试。**

2.2 设计方案与核心技术

2.2.1 整体方案

在 1.4 节作品主要工作中提到，**本作品旨在实现对 Android Native 系统服务的模糊测试，在这个过程中逐步要实现的主要功能就是 Android 系统服务接口文件收集；接口模型预处理与后处理；接口之间依赖关系推断；Fuzzer 设计与实现；Manager 设计与实现。**

基于我们的构思流程，我们把作品实现分为了五个部分，如图 2-5 所示。首先，我们设计一个接口收集器收集目标服务中的所有接口，包括顶级接口和多级接口。然后，设计一个接口模型提取器为每个收集的接口中的每个候选事务提取输入和输出格式以及变量语义，即变量名称和类型。提取器还收集与变量相关的结构、枚举和类型别名的定义。接下来，通过依赖推断器部分推断接口依赖，

以及事务内和事务间的变量依赖。最后，基于上述信息，fuzzer 引擎随机生成事务，并调用相应的接口来模糊本地系统服务。但是，以上技术实现之后，我们还需设计并实现一个能驱动 Fuzzer 在设备上自动运行、确保设备状态、日志同步等工作的模块，本作品设计并实现了 Manager 模块来完成这一功能。这一部分我们将在作品测试与分析中详细阐述。

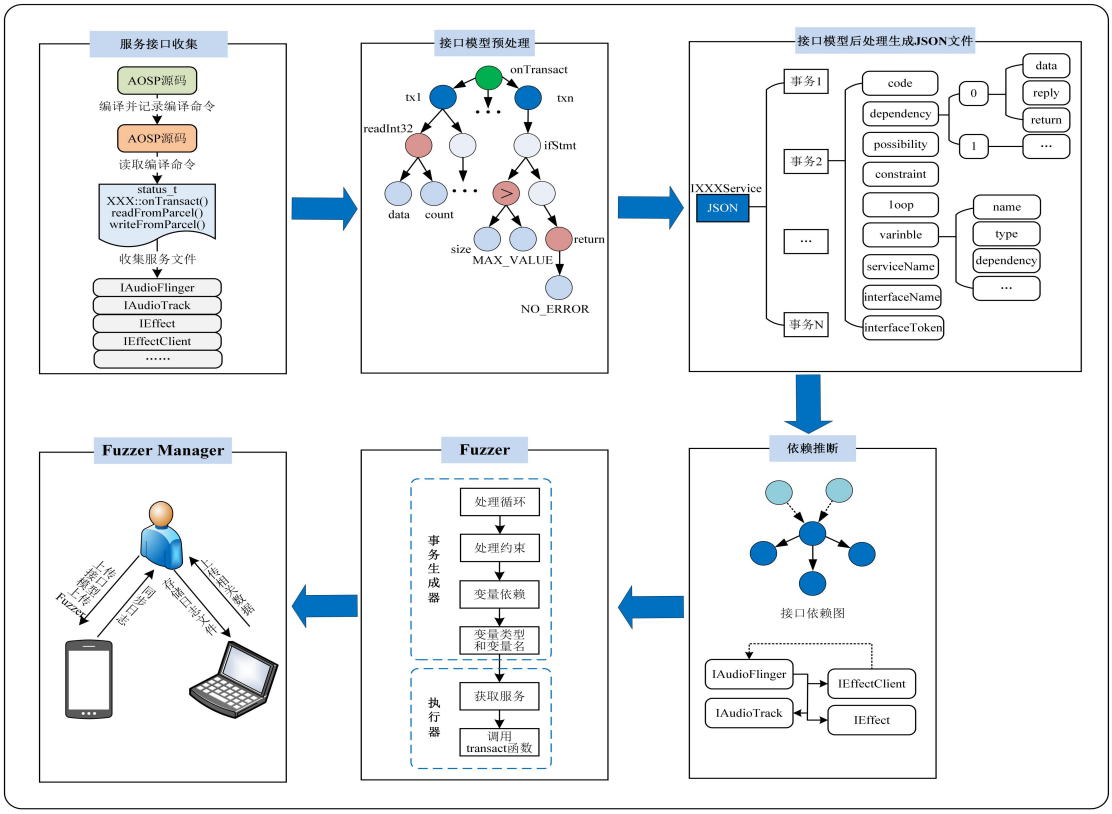


图 2-5 作品整体方案图

因此，也可以衍生出五个部分的实现方案来完成作品，即接口文件收集方案、接口模型提取方案、依赖推断方案、fuzzer 引擎设计方案、Manager 模块设计方案。前三个方案是每个流程技术实现方案，最后两个方案是整个作品运行实现的必要条件，我们将在 2.3 节作品实现部分对其二者进行阐述。

2.2.2 接口文件收集方案

2.2.2.1 记录编译命令

在 Android 本地系统服务的实现中，除了使用 C++源代码编写，还有一部分使用 Android 接口定义语义（Android Interface Definition Language，AIDL）

来实现。这些使用 AIDL 编写的系统服务在 AOSP（Android Open Source Project）编译过程中会经过 AIDL 转换工具的处理，该工具会动态生成对应的 C++文件。为了收集完整的本地服务接口，本作品采用了一种解决方案，即在 AOSP 的编译过程中提取编译命令。**这些编译命令会将每个 C++文件和 AIDL 文件编译生成的结果保存在 AOSP 根目录下的 out 目录中，并以.cpp 或.cc 后缀结尾的文件形式存在。**

换句话说，为了收集由 AIDL 工具生成的接口，**我们可以检查编译命令中列出的每个 C/C++文件，并识别其中是否包含由 AIDL 工具生成的代码。**一旦我们发现这样的文件，我们就可以将其标记为动态生成的接口，并收集相关的信息，如接口名称、方法列表、参数信息等。举个例子，假设我们有一个 Android 应用项目，其中包含一个 AIDL 文件 MyInterface.aidl。当我们编译该项目时，AIDL 工具将自动生成名为 MyInterface.cpp 的 C++文件。在检查编译命令时，我们会发现 MyInterface.cpp 文件作为源代码出现在其中。我们可以通过识别这个文件，并收集其中的接口信息，以获得动态生成的接口信息。这样做可以确保收集到所有通过 AIDL 定义的接口，使得接口文档更加全面和准确。

为了记录 AOSP 编译过程中的编译命令，我们可以在 AOSP 根目录下执行以下命令：

```
make -j12 showcommands 2>&1 > cmd.txt
```

在这个命令中，我们使用了 make 工具来构建整个 AOSP 项目。选项-j12 表示最多使用 12 个线程并行编译，以加快构建速度。`showcommands`选项用于显示完整的编译命令，包括编译器和链接器的详细输出。`2>&1`将标准错误输出重定向到标准输出，`>`符号将标准输出重定向到`cmd.txt`文件中。执行这个命令后，编译过程中的所有命令和输出信息将被记录在`cmd.txt`文件中。这些命令包括编译源代码、链接可执行文件或库文件等操作。通过查看`cmd.txt`文件，我们可以了解整个编译过程中的详细操作和输出，以便进行分析、调试或记录。

请注意，这个命令需要在正确配置和安装了 AOSP 环境的情况下执行，并确保在足够的磁盘空间和系统资源的情况下进行编译。执行时间可能较长，具体取决于硬件性能和项目规模。

2.2.2.2 收集系统服务接口相关的文件

系统服务接口的实现通常会包含一个 onTransact 的函数。基于这个特性，通过搜索包含该函数的文件，我们可以找到与接口相关的代码文件。此外，我们还需要识别包含 readFromParcel 和 writeToParcel 函数的文件。这些函数是 Parcelable 接口的实现方法，表示该类可以被序列化并存储在 Parcel 对象中进行传输。基于以上特性，我们可以设计算法来收集与接口相关的文件，以便后续生成接口模型。具体的算法可以参考表 2-1 所示的算法 1。

表 2- 1 算法 1 服务接口相关文件收集

算法 1 服务接口相关文件收集

```
输入：cmd.txt
输出：service.txt
    cmdFile open("cmd.txt")
    serviceFile open("service.txt")
    for cmdLine in cmdFile do
      if "prebuilts/clang" not in cmdLine then
        continue
      end if
      If "mkdir-p" or "rm-f"in cmdLine then
        continue
      end if
      if cmdLine.filepath in serviceFile then
        continue
      end if
      if line is endWith(".cpp")or endWith(".cc") then
        for contentLine in open(filepath)do
            if ( "onTransact" or "readFromParcel" or "wiriteToParcel" in contentLine) then
              serviceFile.add(filepath)
              break
            end if
        end for
       end if
     end for
```

该收集器算法的目的是识别并记录包含关键函数的文件，以便后续对接口进行分析处理。通过这样的收集过程，我们可以方便地提取系统服务接口的信息并生成相应的接口模型。

2.2.3 接口模型提取方案

由于我们想要系统地模糊测试 Android 原生系统服务，**我们需要获得一套完整的、精确的接口，连同它们的所有事务。**为此我们需要从变量模式、变量名称和变量类型几个方面着手。而且，我们最好找到一个统一的方法来处理 c++中定义的接口和 AIDL 中定义的接口。基于以上原则，我们方案如下分点展述。

2.2.3.1 接口模型预处理

2.2.3.1.1 cc1 编译命令提取

本作品需要使用 Clang 插件来提取接口模型，在 AOSP 编译过程中，使用的就是 Clang 编译器来编译源代码。在 AOSP 编译过程中，使用的是 Clang 编译器来编译源代码。为了提取与系统服务接口相关的文件的编译命令，我们可以通过分析编译过程中生成的编译命令文件（cmd.txt）。

在这个过程中，我们可以从编译命令文件中筛选出与 Clang 编译器相关的命令（ccl 命令）。这些命令包含了 Clang 编译器的选项和参数，用于将源代码编译成目标文件。通过解析编译命令文件，我们可以提取出每个 C++文件的路径，并将其作为键（key）记录下来。同时，我们也会记录对应的 ccl 命令作为该键的值（value）。这样，我们就可以建立一个键值对的映射关系，将 C++文件路径与其对应的 ccl 编译器命令进行关联。其表示形式如下：

{

filepath1: clang 编译器路径 -ccl 编译选项 I 包含的头文件-D 编译选项 -W 编译优化参数 -o 输出文件路径-x c++输入文件路径...

filepath2: clang 编译器路径-ccl 编译选项-I 包含的头文件 D 编译选项 -W 编译优化参数 -o 输出文件路径-x c++输入文件路径...

}

为了更方便地处理和分析系统服务接口相关的文件，我们可以将这些键值对记录在一个 JSON 格式的文件（ccl_cmd.json）中。这个文件会包含多个条目，每个条目表示一个 C++文件及其对应的 ccl 编译器命令。

通过这种方式，我们可以轻松地获取与系统服务接口相关的文件在编译时所运行的Clang编译器的cc1命令。这些命令可以提供给后续的接口模型生成过程，以便进一步分析和处理系统服务接口的特性和行为。

2.2.3.1.2 Clang 插件及使用

1、Clang AST概述

在AIDL中，解决提取接口的一个方案是从AIDL文件中提取接口模型。然而，这种方法会错过在Android源代码中直接用C++实现的大范围接口，这个方法有失完整性。我们可以将AIDL文件转换为C++文件，而且这样转化还不会丢失原来的信息，确保接口的准确性和完整性。

在AIDL文件转换成C++文件后，本作品使用了Clang AST方法[13]来提取接口模型。Clang AST（抽象语法树）是提取接口模型的一个强大表示形式。它提供了代码结构和语义的全面视图。**在AST中，变量名和变量类型完整地保留下来。此外，每个类型转换表达式都在AST中记录，编译器解析所有头文件的依赖关系，并按正确的顺序在AST中提供类型信息。**因此，我们可以按顺序处理AST并解析typedef类型的原始类型。此外，AST还提供了onTransact分派器中每个接口的所有事务代码的清晰要点。AST中的每个语句（例如顺序语句和条件语句）都是单独表示的，这些特性使得从AST表示中提取接口模型变得非常方便。具体来说，通过Clang AST的表示形式，预处理阶段可以获得以下信息：

（1）变量名和变量类型

（2）枚举类型及其值

（3）原始结构体类型[13]

（4）类型别名信息，即typedef的原始类型

2、Clang插件使用

我们不妨通过实例来说明，在本作品中Clang插件主要通过访问VisitXXX函数来提取接口模型，如图2-6所示。

```
1.  bool VisitCXXRecordDecl(CXXRecordDecl *recordDecl)
2.  {
3.     //当访问到类、结构体的声明节点时执行操作
4.       ....
5.       return ture;
6.  }
7.  bool VisitFunctionDecl(FunctionDecl *func)
8.  {
9.     //当访问到函数声明时，执行操作
10.    //例如：onTransact函数，readFromParcel函数，writeToParcel函数
11.    //flatten函数、unflatten函数等等
12.    ...
13.    return ture;
14.  }
15.  bool VisitTypedefDecl(TypedefDecl *typedefDecl)
16. {
17.    //当访问到ypedef声明时，将原始类型和别名记录下来
18.         ...
19.       return ture;
20.  }
21.  bool VisitEnumDecl(EnumDecl *ed)
22. {
23.    //当访问到枚举类型时
24.    //记录枚举类及其所有值
25.    ...
26.    return ture;
27.  }
```

图 2- 6 Clang AST 插件遍历的节点

通过使用 Clang AST，我们可以提取出以下信息：

（1）类或结构体声明：当遍历到 CXXRecordDecl 节点时，即类或结构体的声明，预处理阶段需要保存相关信息，对于 Class 类需要获取其类型名称、命名空间、基类信息等。对于 Struct 结构体需要获取结构体名称、结构体成员等。

（2）函数的定义：当遍历到 FunctionDecl 节点时，说明正在遍历某个函数的声明，预处理需要判断该函数是否为系统服务需要的函数，例如 onTransact 函数、readFromParcel 函数、writeToParcel 函数等。对于上述函数，预处理阶段需要遍历该函数体节点内容，保存 swich-case 语句的相关信息，以及事务 code 值、变量的相关信息、函数调用信息、条件语句和循环语句及返回值信息。

（3）类型别名的定义：当遍历到 TypedefDecl 节点时，即类型别名的定义，预处理阶段需要记录该声明

（4）枚举类型：当遍历到 EnumDecl 节点时，即枚举类型，预处理阶段需要记录枚举类型的所有内容及其常量值。

2.2.3.2 目标接口模型提取

1、事物 code 识别

在目标接口中，“onTransact”函数根据事务码将控制流分派给不同的目标事务。通常，这个分派过程在 C++源代码中使用 “switch” 语句来实现，并在 AST 中转换为多个 “case” 节点。每个 “case” 代表一个要调用的事务。因此，**我们可以通过分析 AST 中的 “case” 节点来轻松识别目标接口的所有事务，并确定相关的事务代码常量，如图 2-7 所示。**

通过分析 AST 中的 “switch” 语句，我们可以找到目标接口的事务代码，每个事务代码对应一个 “case” 节点。**通过分析 “case” 节点，我们可以获取事务的相关信息，如事务码、参数列表等。**这样，**我们就可以识别出目标接口的所有事务，并了解它们的调用方式和参数要求。**

```
1.status t BnDrm:onTransact(uint32_tcode,const Parcel&data,Parcel *reply,
                                    uint32_t flags){
2.    switch(code)
3.        case IS_CRYPTO SUPPORTED:
4.        {
5.            ...
6.        }
7.        case CREATE PLUGIN:
8.        {
9.            ...
10.       }
11.       case DESTROY PLUGIN:
12.       {
13.           ...
14.       }
15.       ...
16. }
17.}
```

图 2-7 onTramsact 函数的 swich-case 语句

2、输入和输出变量提取

在识别事务代码后，我们需要提取每个事务的输入和输出变量，以生成事务接口模型。通常情况下，**输入变量会在客户端被序列化后放入Parcel类的data对象中，**在服务端则使用相应的readXXX函数进行反序列化读取。此外，为了推断不同事务之间的依赖关系以及接口之间的依赖关系，**我们还需要提取输出变量，它们被序列化到Parcel类的reply对象中。**通过分析代码，我们可以找到涉及输入和输出的关键代码片段。**在输入方面，我们可以定位到客户端代码中的序列化操作和服务端代码中的反序列化操作，进而提取输入变量及其相关信息。对于输出方面，我们可以追踪服务端代码中将结果序列化到reply对象的部分，从中提取输出变量。**具体来说，一共需要被提取的变量类型有5种。

（1）顺序变量

顺序变量对应于程序中按照顺序执行的语句，它们的存在不依赖于任何条件。在我们的上下文中，顺序变量通常对应于在服务端代码中使用Parcel类的readXXX函数进行反序列化操作。此外，对于实现了Parcelable接口的类型，我们需要使用重写的readFromParcel函数来进行对象的反序列化操作。这**个函数会根据对象的定义和序列化过程中的数据，逐个读取相应的字段，并将其还原为原始对象。**

如下图2-8所示给出顺序语句在事务中的实例代码：

```
1.  //checkInterfance检查接口是否存在
2.  CHECK INTERFACE(IDrm,data,reply);
3.  //readXXX()
4.  String8 appPackageName data.readString80;
5.  pid_t pid data.readInt320;
6.  //read(var,sizeof(var))
7.  uint8_t uuid[16];
8.  data.read(uuid,sizeof(uuid));
9.  audio config config=);
10. data.read(&config,sizeof(config));
11. //readStrongBinder
12. sp<IDrmClient>listener
13.     interface cast<IDrmClient(data.readStrongBinderO)
14. //readFromParcel(data)
15. android:AudioClient client;
16. client.readFromParcel(&data);
```

图 2-8 顺序变量源码样例

① checkInterface: 在每个事务中的第一条语句通常会调用`checkInterface`函数，这是一个宏定义函数。在客户端进行 RPC（远程过程调用）时，会将`interfaceToken`字符串写入`Parcel`对象中。在服务端接收到调用请求后，首先会检查给定的服务接口的唯一`interfaceToken`是否与调用接口匹配。如果不匹配，则会直接返回`PERMISSION_DENIED`（权限被拒绝）。在模糊测试的输入构造中，为了挖掘更深层的漏洞，并达到更高的代码覆盖率，Fuzzer 应该为每个服务接口构造正确的`interfaceToken`，而不是使用随机生成的字符串。这样可以确保在模糊测试过程中能够正确匹配服务接口，从而执行更多的代码路径和边界条件。

②readXXXX:在第 4 行的代码中，readString8 从 Parcel 类的 data 对象中反序列化一个单字节型字符串，并以 appPackageName 变量接收，显然，这个变量名表示的是一个包名，含有语义信息，Fuzzer 应该更高概率的生成与包名有关的字符串，而低概率的生成随机字符串。在第 5 行中，readInt32 从 Parcel 对象中读取了一个 int32 型变量，并以 pid 变量接收，很容易发现这也是一个含有语义信息的变量，它表示的可能是进程 id,因此 Fuzzer 采取同样的策略来构造语义正确的输入。

③read(var,sizeof(var)):在这种情况下，服务端会从 data 对象中直接拷贝一个数组或者原始结构体。在第 8 行中，服务端直接读取了一个 uit8_t 类型的数组，同样在第 10 行中直接读取一个 audio_config 结构体。

④readStringBinder():在代码的第 13 行中，存在一个函数调用，**它从 data 对象中读取一个 Binder 对象。这意味着在这个事务中可能会使用到其他系统服务接口的函数。**通过观察这个函数调用，我们可以推断出当前服务接口可能存在对其他服务接口的依赖关系。

⑤readFromParcel(&data):这条顺序语句相对于上述的比较特殊，因为它的反序列化操作存在于实现 Parcelable 接口类或结构体中，这也是为什么在服务相关文件收集阶段需要收集含有 readFromParcel 函数的文件。

（2）条件变量

条件变量在程序中对应着条件语句，例如 if-else 语句或者 switch-case 语句。在代码示例中，我们可以看到对于输入变量 mime，它的生成和构造取决

于变量 flag 的值。**当 flag 满足特定条件时，我们需要构造变量 mime 的值；反之，如果条件不满足，则可以跳过对 mime 的构造过程，如图 2-9 所示。**

在事务接口模型中，**即使变量 mime 是一个条件变量，我们仍然需要记录下它作为输入变量的存在。**这是因为在模糊测试中，我们需要尽可能地覆盖不同的代码路径和情况，包括满足和不满足条件的情况。通过记录和使用条件变量，我们能够更全面地生成模糊测试用例，以发现可能存在的漏洞和问题。

```
1. const char *mime = NULL;
2. const int32_t flag=data.readInt320;
3. if (flag != 0) {
4.   mime = data.readCString();
5. }
```

图 2-9 条件变量源码样例

（3）循环变量

循环变量对应于程序中的循环语句，例如常见的 for 循环和 while 循环。在接口模型分析中，我们需要处理循环变量的情况。

如下图 2-10 所示，我们可以看到 key、fd、value 这三个变量存在于命名为 loop0 的循环体内。在事务接口模型中，我们需要记录循环体的执行次数（如 size）以及相关的循环变量。我们以 for 循环为例。

（4）返回值变量

返回值变量对应于程序语句中的 return 语句，在一个事务中可能会存在多个 return 语句，也就意味着该事务存在多种可能的执行路径，如图 2-11 所示，在第 4 行和第 7 行都有 return 语句，如果输入变量 num Bytes 的值大于 MAX BINDER_TRANSACTION_SIZE 的话，该事务会直接在 reply 中写入一个 BAD_VALUE 并且直接返回 DRM_NO_ERROR,在提取事务接口模型中，需要记录不同的返回路径和不同路径下的输入变量。

```
1.  const int size data.readInt320;
2.  for (int index =0;index size;++index){
3.        if(!data.dataAvailO){
4.              break;
5.        }
6.        const String8 key(data.readString80);
7.        if(key ==String8("FileDescriptorKey")){
8.           ...
9.           int fd data.readFileDescriptor  ()  ;
10.          ...
11.
12.     } else {
13.          const String8 value(data.readString8  ()  ;
14.          ...
15.     }
16. }
```

图 2- 10 循环变量源码样例

```
1.  const uint32_t numBytes=data.readInt32();
2.  if(numBytes MAX_BINDER TRANSACTION_SIZE){
3.        reply->writeInt32(BAD_VALUE);
4.        return DRM_NO_ERROR;
5.  }
6.
7.  return DRM_NO_ERROR;
```

图 2- 11 返回值变量源码样例

（5）类型定义提取

①类结构体类型提取

这类类型指的是联合和结构体。在 AST 中，我们可以轻松地提取这些对象的成员信息。

②枚举类型提取

通过分析 AST，我们可以识别和提取枚举类型的定义及其包含的所有常量。每个枚举常量都具有唯一的名称和对应的值，这些值可以用于构造输入变量时的合法取值范围。

③类型别名提取

在服务端的源代码中，有些变量可能使用了通过 typedef 定义的类型别名，而 Fuzzer 在构造输入时需要了解这些变量的实际原始类型。因此，我们需要记录每个类型的别名以及其对应的实际类型。

详细源码样例如图 2-12 所示

通过以上步骤，便提取了所有系统服务的接口模型，并以 JSON 格式存储，如图 2-13 所示。

2.2.4 依赖推断方案

2.2.4.1 变量依赖

变量依赖可以根据是否在同一个事务内分为事务内依赖和事务间依赖。

（一）事务内依赖

在同一个事务中，变量之间可能存在依赖关系，包括条件依赖和循环依赖。条件依赖意味着一个变量的生成取决于另一个变量的值。例如，前文所提的变量 mime 的生成取决于变量 flag 的值。如果 flag 等于 0，Fuzzer 将跳过生成 mime 变量；反之，如果 flag 不等于 0，Fuzzer 将生成 mime 变量。循环依赖指的是一个变量的读写次数取决于另一个变量。也例如，变量 size 和 key 之间存在循环依赖关系。

（二）事务间依赖

在模糊测试中，事务间依赖变量指的是一个事务中的输入变量依赖于另一个事务中的输出变量。换句话说，一个事务的输入可以通过另一个事务的输出来获得，对于事务间的依赖，本文通过表 2-2 所示的算法 2 实现。

```
1.  struct audio_config{
2.          uint32_t sample_rate;
3.          audio_channel_mask_t channel_mask;
4.          audio_format_t format;
5.          audio_ffload_info_t offload_info;
6.          uint32_t frame_count;
7.  };
8.  enum {
9.          AUDIO INTERLEAVE LEFT=0,
10.         AUDIO INTERLEAVE RIGHT=1,
11. };
12. typedef struct audio_config audio_config_t;
13. typedef int__kernel_pid_t;
14. typedef__kernel_pid_t __pid_t;
15. typedef__pid_t pid_t;
```

图 2- 12 类型定义源码样例

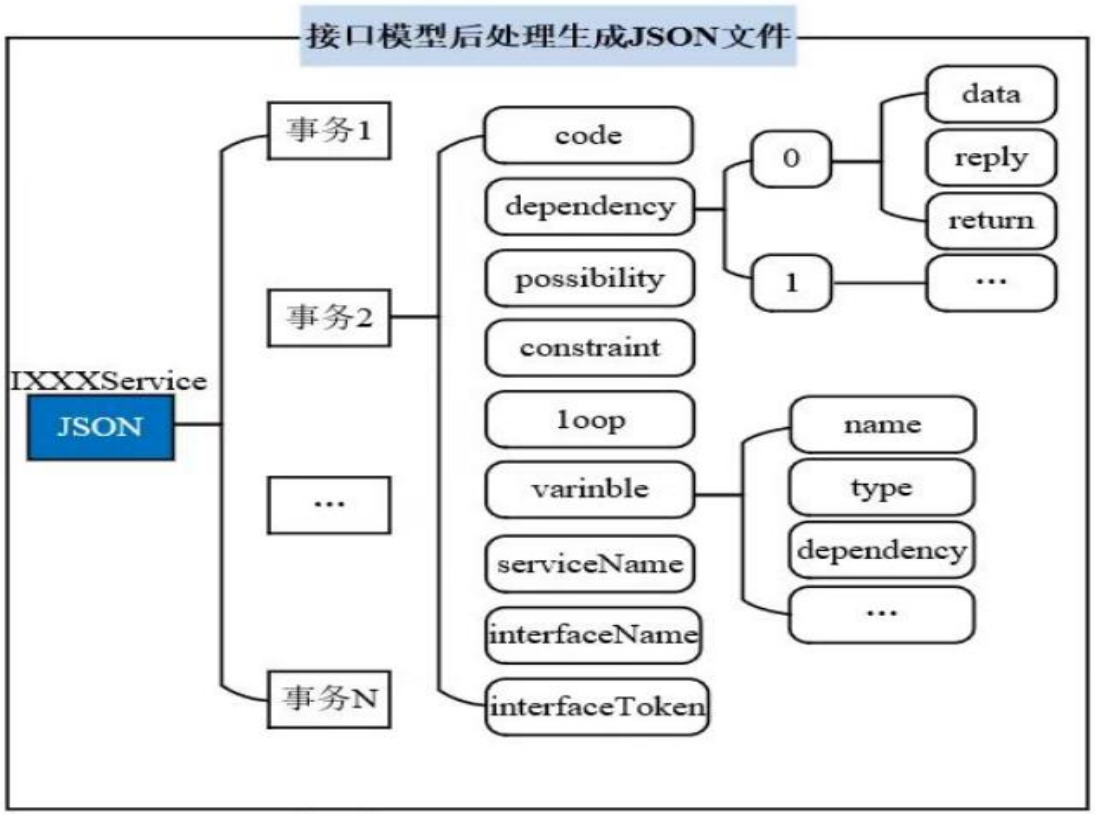


图 2- 13 系统服务接口 JSON 格式图

表 2- 2 算法 2 事务间变量推断器

算法 2 事务间变量推断器

```
输入：接口模型(M)
输出：事务间依赖关系组(D)
    D={}
    INPUT=[ ]     //输入变量
    OUTPUT=[ ]   //输出变量
    for var in M do
      if var is input then
        add var into INPUT
      end if
      if var is output then
        add var into OUTPUT
       end if
    for iVar in INPUT do
       for oVar in OUTPUT do
         if iVar.txCode !oVar.txCode then
           if iVar.type ==oVar.type then
             if iVar.type is complex then
               add (iVar,oVar) into D
             else if iVar.name is similar to oVar.name then
               add (iVar,o Var) into D
             end if
           end if
         end if
       end for
    end for
```

事务间依赖变量，确定它们之间的关系具体规则如下：

（1）分别为提取事务的**输入变量和输出变量**。

（2）计算输入变量名和输出变量名之间的相似度。相似度度量算法可以根据需求进行自定义，例如基于字符串匹配或基于语义分析等。

（3）**根据相似度度量结果，确定输入变量和输出变量之间的依赖关系。**如果相似度高于设定的阈值，可以认为它们之间存在依赖关系。

（4）输入变量类型和输出变量类型相同。

这样，在模糊测试中生成的输入可以更加准确地模拟实际系统中的依赖关系，提高测试的覆盖率和真实性。

需要注意的是，**相似度度量算法的选择和参数设定对于准确识别依赖关系至关重要。**根据具体情况和需求，可以选择合适的相似度度量算法，并进行参数调优，以获得最佳的依赖关系识别效果。

2.2.4.2 服务接口依赖

接口依赖可以分为两种，生成依赖和使用依赖。

（1）生成依赖

生成依赖是指一个接口可以通过另一个接口的调用生成所需的数据或对象。在 Android 原生系统服务中，我们可以直接从服务管理器获取顶层接口，这些接口提供了基础的系统功能。而对于多层接口，我们观察到上层接口在调用过程中会使用`writeStrongBinder`将一个深层接口序列化为回复(`reply`)对象。

通过这个观察，我们可以轻松收集接口之间的生成依赖关系。具体而言，我们可以识别出上层接口调用了哪些深层接口，并将这些依赖关系记录下来。这样，我们就能够建立起接口之间的生成依赖关系图。

通过分析生成依赖关系，我们可以更好地了解系统中接口之间的依赖关系，为模糊测试提供更准确的输入构造。例如，当需要模糊测试一个深层接口时，我们可以首先构造生成它所需的上层接口调用，以确保模糊测试的输入满足生成依赖关系。

（2）使用依赖

接口间依赖推断，我们通过表 2-3 所示的算法 3 实现。

在服务接口中，存在着一种依赖关系，即一个接口可能会使用到另一个接口中的服务。我们将这种关系称为服务接口间的使用依赖。通过研究源代码，**我们发现当一个接口 A 需要使用另一个接口 B 时，它会在 Parcel 类的 data 对象中使用 readStrongBinder 函数方法来反序列化获取服务接口 B。**通过分析 readStrongBinder 的调用情况，我们可以识别出一个接口是否依赖于另一个接口。

这种使用依赖的分析对于了解系统中各个接口之间的依赖关系非常重要。它可以帮助我们理解接口的功能和相互关联，进而在模糊测试中生成更准确、合理的输入。通过识别使用依赖，我们可以构造正确的接口调用序列，以满足接口间的依赖关系，从而提高模糊测试的覆盖率和有效性。

表 2- 3 算法 3 接口间依赖推断

算法 3 接口间依赖推断

```
输入：事务接口模型(M)
输出：接口间依赖关系(D)
    D={}
    INPUT=[ ]
    OUTPUT=[ ]
    for var in M do
      if var is input then
        add var into INPUT
      end if
      if var is output then
        add var into OUTPUT
      end if
    end for
    for oVar in OUPUT do
      if o Var.type is IBinder then
       if oVar.interfaceName !oVar.type.interfaceName then
         add (oVar.interfaceName,oVar.type.interfaceName,gen_dependency) into D
       end if
      end if
    end for
    for iVar in INPUT do
      if i Var.type is IBinder then
        if iVar.interfaceName !=iVar.type.interfaceName then
          add (iVar.interfaceName,iVar.type.interface Name,use_dependency) into D
        end if
      end if
    end for
```

在如下图 2-14 部分服务接口依赖图所示，本文把服务接口分为三类，第一类是顶级服务接口，即是注册在 ServiceManager 中的实名服务，该类接口是由 Android 系统进行初始化，不依赖于其他接口，例如图 2-14 所示的 IMediaPlayerService 。第二类是通过顶级接口的生成依赖生成的匿名服务,例如图 2-14 所示的 IGraphicBufferProducer 需要通过两层依赖关系才能生成。还有一类接口既不属于顶级服务接口，也不属于多级服务接口，例如图 2-14 所示的 IRemoteDisplayClient 、IMediaPlayerClient 、IStreamSource 等，本文把该类接口称为定制化服务接口，需要用户自行实现和生成。

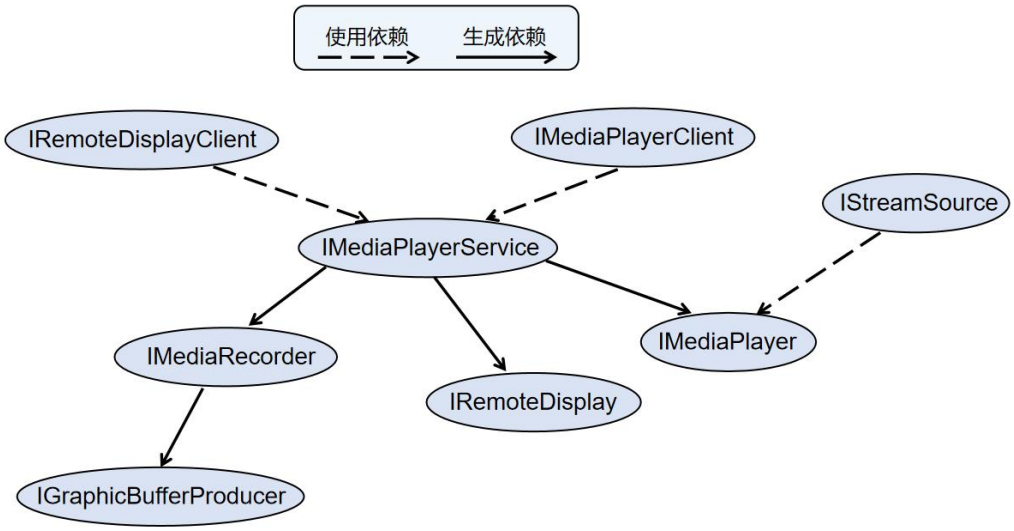


图 2- 14　部分服务接口依赖图

2.3 作品实现

本作品基于 2.2 节中的全部技术，设计 Fuzzer 引擎随机生成事务，并调用相应的接口来模糊本地系统服务，这样就可以串联完成整个作品测试流程。但是，我们还需设计并实现一个能驱动 Fuzzer 在设备上自动运行、确保设备状态、日志同步等工作的模块，本作品设计并实现了 Manager 模块来完成这一功能。

2.3.1 Fuzzer 引擎

Fuzzer 中包含事务生成器、执行器两部分。

2.3.1.1 Fuzzer 流程概要

Fuzzer 设计和实现是模糊测试的关键部分，它是模糊测试的执行引擎。在 Fuzzer 的实现中，根据之前准备的事务接口模型规范，它会生成测试样例并将其输入到待测的目标服务中。换句话说，Fuzzer 根据事务接口模型规范来构造有效的测试输入，以验证目标服务的安全性和稳定性。其工作流程如下图 2-15 所示：

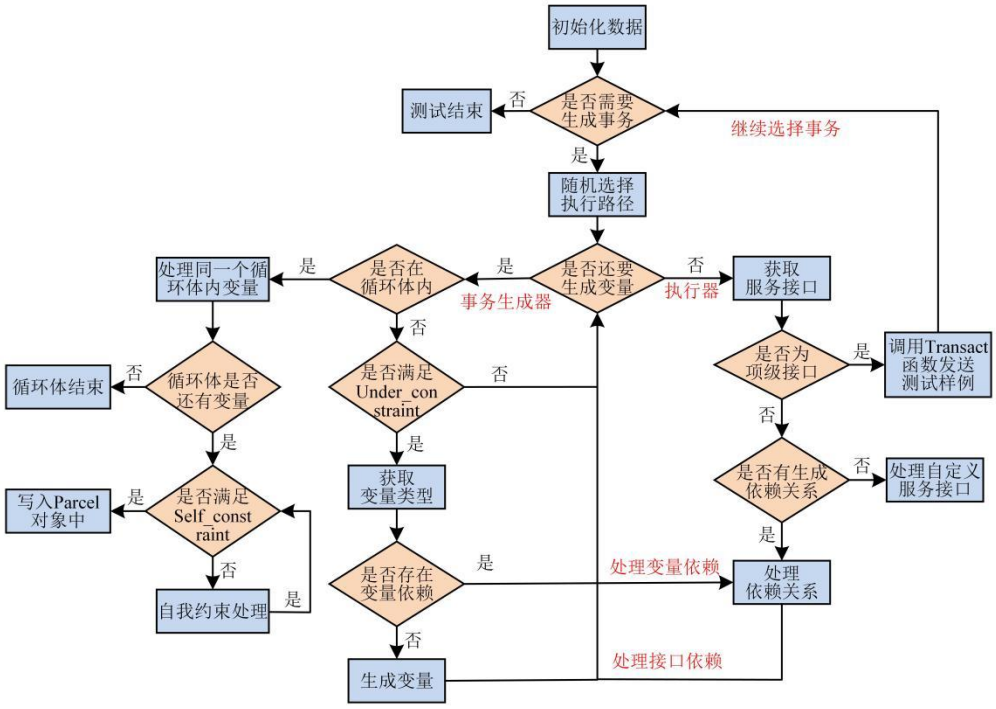


图 2- 15 Fuzzer 流程图

2.3.1.2 事务生成器

为了确保对每个本地服务接口中的每个事务进行模糊测试，Fuzzer 采用迭代测试的方法，逐个遍历每个接口中的事务。而在每个事务中，由于存在多个 return 语句和不同的执行路径，Fuzzer 需要随机选择执行路径以增加测试的多样性。然而，考虑到大多数事务中的第一条语句是 CHECKINTERFACE()宏函数并含有 return 语句，**这样的执行路径在接口模型提取过程中已经被记录下来，因此为了提高模糊测试的效率并减少不必要的执行时间，Fuzzer 不会选择这条执行路径进行测试。**

另外，**Fuzzer 需要根据事务接口模型逐一生成特定事务的输入变量。在变量生成过程中，Fuzzer 遵循以下原则：**

（1）循环第一

当生成输入变量时，首先判断该变量是否在循环体内，如果该变量在循环体内，例如 loop0 循环，则遍历找到与该变量在同一个循环体内的所有变量，因为同一循环体内的变量 Fuzzer 要产生相同的数量，以此满足输入的语法要求。

表 2- 4 算法 4 自我约束（self_constraint)处理

算法 4 自我约束（self_constraint)处理

```
输入：变量(var)
输出：变量值(value)
    self_constraint var.self constraint
    opcode self constraint.opcode
    if "==" in opcode then
        value self constraint.rhs.value
    else if opcode=" ！" then
        value =0
    else if opcode="" then
        value 1
    else then
        //检查是否满足自我约束
        while checkSelfConstraint(self constraint) is false do
            value generate Value ()
            store Value(value)
        end while
    end if
```

（2）约束第二

在生成单个变量时，Fuzzer 首先要确保满足事务接口模型中的约束条件，特别是针对条件变量。**这意味着变量的生成需要满足特定的约束条件才能进行。此外，Fuzzer 还需要考虑变量自身的约束条件，即自我约束。**如果变量本身具有约束条件，**Fuzzer 会使用表 2-4 所示的算法 4 来满足这些约束条件**。这种策略能够覆盖更深层次和更多的代码路径，从而提高模糊测试的漏洞发现能力。

（3）依赖第三

当一个变量可以由其他事务生成时，即该变量依赖于其他事务，Fuzzer 会按照较高的概率选择生成它。在这种情况下，**Fuzzer 首先需要生成依赖的事务，再通过依赖事务输出的 Parcel 类的 reply 对象中读取该变量值，该变量值为系统生成而不是随机生成，为系统输入提供了更符合语义的输入，能挖掘更深层次的漏洞。**然而，对于概率较低的情况，Fuzzer 不会严格遵循这一原则。

2.3.1.3 执行器

一旦 Fuzzer 构造好特定事务的输入后，执行器就会负责执行对目标服务的

模糊测试。**执行器首先需要获取目标服务的接口，然后调用统一的transact(code, data, reply, flag)函数进行调用。在这个过程中，有三种情况需要考虑。**

表 2-5 算法 5 执行器处理

算法 5 执行器处理

```
输入：事务（tx)
输出：执行结果（ret)
    Sm=defaultServiceManager()  //获取ServiceManager
    Binder=NULL
    If generatedInterface.find(tx.interfaceName)!=NULL then
        binder =generatedInterface[tx.interface Name]
        if binder is alive then
        return binder
        end if
    end if
    if tx.serviceName!= "None" then
        Binder= sm.checkService(tx.serviceName)
    else if tx.dependency size()!=0 then
        DependencySolver.solve () //处理依赖关系
        binder =solve.binder
    else then
        binder=generateInterfaceManually(tx.interface Name) //生成自定义服务接口
        generatedInterface[tx.interfaceName]binder
    end if
       ret= binder.transact(tx.code,tx.data.tx.reply.tx.flags)
```

首先，**如果目标服务是实名服务**，也就是顶级服务接口，执行器可以直接从ServiceManager 中获取该服务的实例，并调用该接口的 transact 函数来进行特定事务的测试。

其次，**如果目标服务是匿名服务接口，**也就是没有具体的服务名称，在ServiceManager 中无法直接获取。对于这种情况，Fuzzer 需要先生成依赖的事务，并从 Parcel 类的 reply 对象中获取到匿名服务接口的实例。然后，同样调用 transact 函数对特定事务进行模糊测试。

最后，**如果目标服务既不是实名服务也不是匿名服务**，即无法通过ServiceManager 获得，也无法通过依赖关系获得，Fuzzer 只能自定义该服务接口并进行实例化。一旦获取到该服务接口的实例，就可以调用 transact 函数对特定事务进行模糊测试。

总处理流程如表 2-5 所示的算法 5，通过这样的处理方式，Fuzzer 能够对目标服务的特定事务进行模糊测试。

2.3.2 Manager 模块

Manager 板块是驱动 Fuzzer 在设备上自动运行、确保设备状态、日志同步等工作的模块。

2.3.2.1 检测设备环境准备

为了让 Fuzzer 能够在真实设备上运行，Manager 需要进行一些准备工作。以下是具体的准备工作：

(1) 确保设备状态：首先，需要确保设备处于正确的状态。设备有四种可能的状态，包括 offline、device、fastboot 和 recovery。offline 表示设备与主机断开连接，device 表示设备已连接，fastboot 表示设备处于快速启动状态，recovery 表示设备处于恢复模式。在进行测试之前，需要确保设备处于正确的状态。

(2)确保 HWASan 状态：为了检测 Native 层 C++代码中的内存错误，本文使用了 HWAddressSanitizer（HWASan）工具。在编译 AOSP 时，会生成使用 HWASan 插桩的共享库。Manager 会检查设备中是否存在 HWASan 相关文件夹，如果不存在，则会重新刷入使用 HWASan 编译的镜像。

(3)确保 root 权限：在对 Android Native 系统服务进行模糊测试时，系统服务对用户的执行权限要求较高。因此，设备需要获得 root 权限才能进行更全面的测试。Manager 会通过 adb 命令获取设备的 root 权限，如果获取失败，则无法进行测试。

(4)准备工作：Manager 会执行以下准备工作：首先，检查设备中是否有正在运行的 Fuzzer 程序，并将其关闭。然后，将 Native 系统服务接口模型和编译好的 Fuzzer 程序推送到设备中。接着，清除设备上存在的 logcat 日志。最后，初始化日志文件夹，在主机和设备中创建用于存储日志文件的文件夹。具体来说，包括 Manager 生成的日志、Fuzzer 运行的日志、logcat 工具生成的日志以及设

备崩溃时产生的 tombstone 日志。

通过以上准备工作，Manager 能够为 Fuzzer 在真实设备上的运行提供必要的环境和条件。这些准备工作确保了设备处于正确的状态，并提供了所需的权限和资源，以便进行有效的模糊测试。

2.3.2.2 Fuzzer 运行

完成上述准备工作后，Manager 将启动多个 Fuzzer 程序（例如 5 个，具体数量可以根据需求自定义）以提高模糊测试的速度和效率。这样可以同时执行多个 Fuzzer 实例，使测试能够并行进行。同时，Manager 还会启动一个 logcat 程序，用于记录设备的状态信息和日志输出。

通过同时运行多个 Fuzzer 程序，可以充分利用系统资源，加快测试的执行速度。每个 Fuzzer 程序都会独立地生成测试样例并将其输入到目标服务中进行模糊测试。同时，logcat 程序会记录设备的状态变化、日志信息以及可能的错误和异常情况，以便后续的分析和调试。

通过并行运行多个 Fuzzer 程序和记录设备状态的方式，Manager 能够更高效地进行模糊测试，提高测试的覆盖率和漏洞发现能力。这种策略可以加快测试过程，并帮助快速定位和排查可能的问题，从而提升整体的测试效率。

2.3.2.3 Fuzzer 工作总结

下面是 Fuzzer 在模糊测试过程中的作用和功能的详细说明：

（1）生成测试用例：Fuzzer 根据事先定义的事务接口模型规范，自动生成符合语义的测试用例。这些测试用例包含各种输入数据和操作序列，旨在模拟真实环境中的各种情况和异常情况。

（2）注入模糊数据：Fuzzer 能够生成各种模糊数据，并将其注入到测试用例中。这些模糊数据可能包含随机、无效、边界值等不同类型的数据，以探测系统服务对异常输入的容错性和安全性。

（3）探索边界条件：Fuzzer 通过构造具有边界值的测试用例，探索系统服务在极限情况下的行为。它能够检测系统对于大量数据、极端输入或异常情况的

处理能力，以及可能导致系统崩溃或漏洞暴露的边界条件。

（4）模拟攻击场景：Fuzzer 能够生成模拟的攻击场景，通过注入特定的恶意数据或操作序列，以测试系统服务的安全性和抗攻击能力。它可以模拟各种攻击类型，如缓冲区溢出、拒绝服务攻击等，以帮助发现潜在的安全漏洞。

（5）监控系统状态：Fuzzer 能够通过 Manager 板块监控系统服务的运行状态和性能指标，如内存使用、CPU 占用等。通过监控这些指标，可以评估系统服务在面对大量测试用例时的性能表现，并发现潜在的资源泄露或性能问题。

（6）收集测试结果：Fuzzer 的 Manager 板块会收集测试过程中产生的日志、崩溃报告、异常情况等信息，以便后续分析和修复。

综上所述，Fuzzer 在 Android 系统服务的模糊测试中扮演着生成测试用例、注入模糊数据、探索边界条件、模拟攻击场景、监控系统状态和收集测试结果等重要角色，帮助发现潜在的安全漏洞和性能问题，提高系统服务的稳定性和安全性。

2.4 小结

本章节首先介绍了本作品的相关技术、原理和工具，如模糊测试技术（Fuzzing）、Binder 通信机制等，为后续的设计方案提供了理论支持。本作品将整体设计方案分为接口文件收集方案、接口模型提取方案、依赖推断方案、Fuzzer 引擎设计方案和 Manager 模块设计方案。前三个方案是每个流程技术实现方案，最后两个方案是整个作品运行实现的必要方案。接口文件收集方案通过记录编译命令收集文件，再通过 onTransact 函数找到文件中与接口相关的代码文件。接口模型提取方案通过 Clang AST（抽象语法树）的方法来提取有效的语义信息并生成 JSON 格式的事务接口模型，此方法能够完整保存变量名、变量类型等接口完整信息。依赖推断方案首先分析了变量依赖的事务内依赖在 Clang AST 节点时提取，事务间的依赖关系通过表 2-2 所示的算法 2 和推断规则来确定。服务接口依赖的生成依赖只能依赖于实名服务生成，使用依赖通过 Parcel 类特性来推断。Fuzzer 通过事务生成器产生符合条件的测试用例，通过执行器将测试用例输入到待测的目标服务中。Manager 板块推动 Fuzzer 在设备上的自动运行、确保设备状态、日志同步等。

第三章 作品测试与分析

3.1 测试方案

本作品的优势性主要体现在三大主要功能的实现，分别是（1）生成 Native 系统服务接口模型；（2）依赖关系推断；（3）自动化检测与结果提取。其中第三个功能本作品设计了 Manager 模块来推进作品测试，**如果运行结果符合预期，则其执行的功能在运行过程中就得到了完整地检测，无需再测试。本部分测试方案主要针对生成 Native 系统服务接口模型与依赖关系推断这两大功能进行功能性测试。**

3.1.1 生成 Native 系统服务接口模型测试方案

主要测试目标：验证该功能所生成的 Native 系统服务接口模型是否符合预期情况，**从而确定接口模型生成功能的准确性、完整性和可靠性**。

本作品在对 Native 系统服务接口生成的过程中，对其进行功能性测试主要通过以下步骤：

（1）单元测试：设计一组测试用例，包括不同类型的接口（包括包含不同方法和参数的接口）。

考虑接口的各种边界条件和特殊情况，以确保接口提取功能的全面性和正确性。首先针对每一种类型的变量（顺序、条件、循环、返回值和类型定义）分别设计并实现一组模拟服务接口。然后利用工具对这些模拟服务接口进行模型提取。

（2）集成测试：单元测试之后，可以进行集成测试，以验证工具能否有效处理复杂交互的服务接口。可以通过设计并实现一些模拟服务接口，这些服务的功能调用取决于其他服务的结果，或者它们可能具有复杂的控制流程。对这些服务接口进行模型提取，以检查我的工具是否能够准确地描述服务的行为，并准确地描绘各服务接口之间的依赖关系。

（3）边界条件和特殊情况测试：我针对边界条件和特殊情况设计了一些特殊的服务接口。例如，服务可能具有大量的函数，函数可能具有大量的参数，或

服务可能具有深层次的函数调用链。可以验证工具在处理这些复杂场景时能否正确地提取服务模型，同时不产生性能问题或错误。

（4）验证生成结果：在所有测试完成后，对比工具生成的接口模型和原先设计的模拟服务接口，检查是否存在差异。测试目标是确保工具生成的模型精确地反映了我原本设计的服务接口。如果存在差异，会进一步调试工具以找出问题所在。

人工执行完上述步骤后，即可判断最终结果是否符合预期情况来衡量Native 系统服务接口生成功能的准确性、完整性和可靠性。

3.1.2 依赖关系推断测试方案

主要测试目标：测试依赖关系推断功能的准确性和完整性。通过对依赖关系推断部分进行测试，**观察所推断的依赖关系是否能够被准确地反映。**

其功能性检测步骤与生成 Native 系统服务接口检测方法类似，但又有不同。步骤主要如下：

（1）针对生成依赖的测试：设计并实现一系列服务接口，这些接口中部分接口通过调用其他接口来生成所需的数据或对象。为了全面覆盖生成依赖的各种可能性，测试接口设计应涵盖单层生成依赖、多层生成依赖以及互相生成依赖等不同情况。运行接口模型生成工具，将设计的服务接口转化为模型，并对生成依赖进行分析。

（2）针对使用依赖的测试：同样地，创建一批服务接口，其中一部分接口会利用到其他接口的服务。为全面测试各种可能的使用依赖，我们需要设计覆盖单个接口的使用依赖、多个接口的使用依赖以及互相使用依赖等各种情形的接口。同样，利用接口模型生成工具对这些服务接口进行模型提取，并对使用依赖进行分析。

（3）针对混合依赖的测试：为全面考察接口模型生成工具的处理能力，设计一些同时包含生成依赖和使用依赖的服务接口。这些接口设计应在保证合理的同时，涵盖尽可能多样化的依赖模式。运行接口模型生成工具，提取服务接口为模型，并同时对生成依赖和使用依赖进行分析。

（4）验证推断结果：将上述每一类依赖的分析结果与预设的依赖关系进行比对，这样可以全方位地验证接口依赖关系推断的准确性。如果结果与预设的依赖关系完全匹配，那么说明我们的依赖关系环节在该测试用例上的表现是成功的。若发现差异，我们需要对工具进行进一步的调试和优化，以提高其精确度。

整个测试方案旨在确保我们的接口依赖关系推断准确无误，以便在后续的工作中更加有效地进行模糊测试。

3.2 测试环境及设备

本文采用的环境如下：

1、AOSP 源代码(Android13.0.0_r4)

2、AOSP 编译输出及使用 HWASan 编译的输出

3、编译环境 Ubuntu18.04

4、真实手机设备运行环境(Pixel4xl)

5、硬件环境使用的是 CPU32Gi7-11800H,SSD2TB

由于本文实现的是对 Android Native 系统服务的模糊测试，因此需要对 Android 原生系统进行测试，通过 Android 开源项目官网 4 下载完整的 AOSP 源码及相应设备（本作品使用的是 Pixel4xl)的二进制驱动文件。

3.3 功能测试结果及分析

3.3.1 Native 系统服务接口提取与依赖推断

本作品测试过程中共收集了 118 个系统服务，其中包括 41 个实名服务，77 个匿名服务。提取的接口名称、方法列表、参数信息等确实准确无误，均本作品体现接口提取器功能的良好运行。其如下图 3-1 所示。

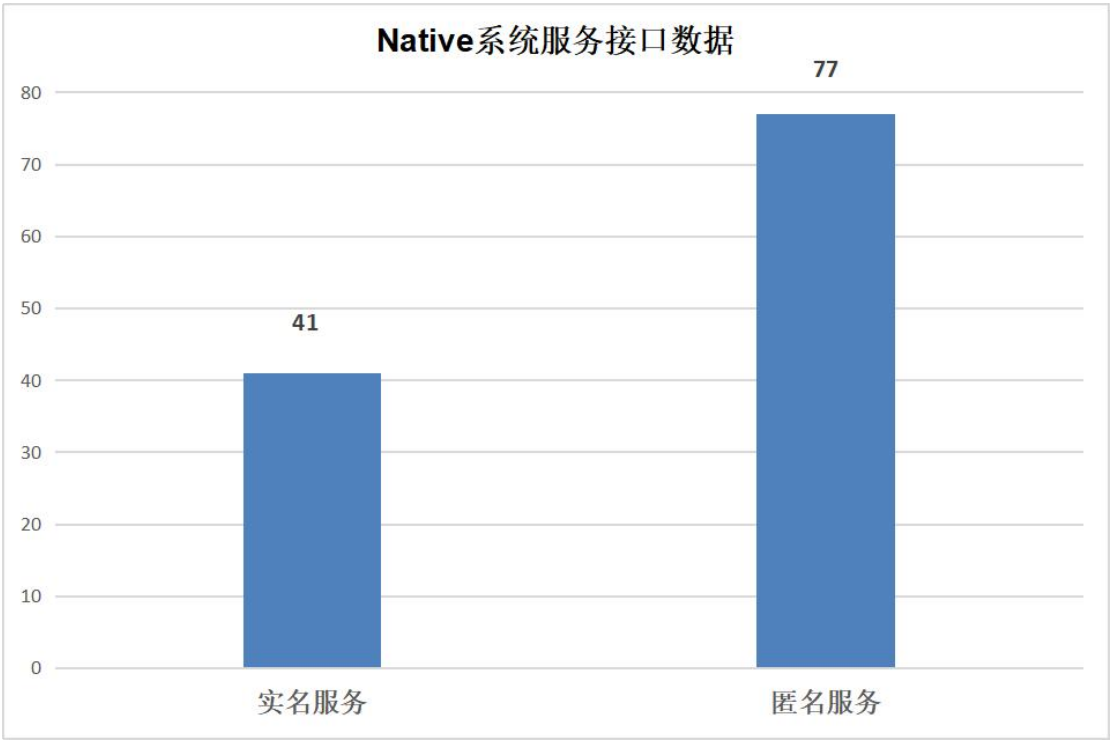


图 3- 1 Native 系统服务接口数据柱状图

此外，本文通过表 2-3 所示的算法 3 来对这些服务接口之间进行依赖推断来生成依赖推断图.在接口依赖图中，本文将服务接口分为三类。第一类是顶级服务接口，这些接口是由 Android 系统进行初始化并注册在 ServiceManager 中的实名服务。这些接口不依赖于其他接口，例如 图 2-14 所示的 IMediaPlayerService 。

第二类是通过顶级接口生成的匿名服务接口。这些接口依赖于顶级接口以及其他多级接口，需要通过多层依赖关系才能生成。例如图 2-14 所示的 IGraphic BufferProducer 接口就需要通过两层依赖关系才能生成。

第三类是定制化服务接口，它们既不属于顶级服务接口，也不属于多级服务接口。这些接口是用户根据自身需求自行实现和生成的，不依赖于系统提供的接口。例如图 2-14 所示的 IRemoteDisplayClient 、IMediaPlayerClient 、IStreamSource 等接口就属于定制化服务接口。

通过将服务接口分为这三类，可以更清晰地描述接口之间的依赖关系和生成方式。顶级服务接口是系统初始化的一部分，而匿名服务接口则是通过多级依赖关系生成的，定制化服务接口则是用户自行实现的。这样的分类有助于更好地理解和分析系统中的接口依赖关系。

3.3.2 事务接口模型

（一）测试结果

本实验中总共提取了 1313 个事务的接口模型，其中 511 个来自于实名服务，802 个来自于匿名服务，如下图 3-2 所示。

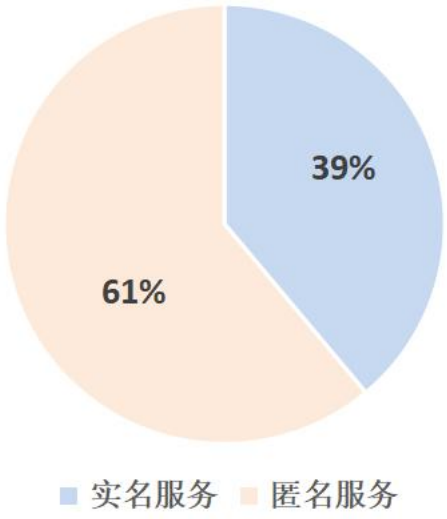


图 3-2 事务接口模型数据图

（二）测试分析

事务接口模型中的提取，主要利用了开源的 Clang AST 插件，Clang AST 很好的保留了源代码的特性，因此本文可以提取完整和精确 C++源代码信息。分析如下：

（1） 使用 onTransact 函数：本文利用 onTransact 函数的特性来识别不同的事务代码。通过分析 onTransact 函数中的 switch-case 语句，可以提取出所有的事务代码，每个 case 对应一个特定的事务。

（2） 利用 Clang AST 插件：本文使用开源的 Clang AST 插件来分析源代码，并提取出事务接口模型所需的信息。Clang AST 插件能够保留源代码的特性，因此可以提取完整和精确的 C++源代码信息。

（3） 输入和输出向量的提取：通过遍历每个事务中的 readXXX 反序列化函数和 writeXXX 序列化函数，可以获取输入和输出向量的相关信息。这些函数用于在客户端和服务端之间进行数据的序列化和反序列化操作。

（4） 变量名和变量类型的利用：提取的变量名和变量类型用于生成特定类

型的变量。如果变量名具有语义信息，Fuzzer 可以生成符合语义的变量值，从而提高生成的输入数据的准确性和合理性。

（5） 枚举类型的提取：本文利用 Clang AST 插件提取了包含在 onTransact 函数中的枚举类型及其常量值。这些信息可用于生成符合语义的变量值，并确保生成的输入数据与枚举类型的定义相符。

（6） 结构体和类型别名的提取：Clang AST 插件可以提取 struct 结构体和 typedef 类型别名的信息。这些信息可用于生成结构体输入变量和使用类型别名的输入变量，以便更准确地构造输入数据。

（7） 路径和代码覆盖率的考虑：根据事务函数中不同的 return 语句，可以产生不同的代码路径，从而实现更全面的代码覆盖率。这些路径和代码覆盖率的信息被记录在事务接口模型中，以便进一步分析和测试。

综上所述，通过利用 Clang AST 插件和相应的技术手段，本文可以提取完整和精确的事务接口模型，包括事务代码、输入输出向量、枚举类型、结构体和类型别名等信息。这些信息对于后续的模糊测试和分析非常有价值，并能提供更全面和准确的接口模型。

3.4 作品测试实验结果

本文针对 Android Native 系统服务进行模糊测试，输入为符合接口语义的测试用例，输出即为崩溃(crash)日志文件，因此可以通过日志文件来得到实验结果，本文中产生的日志文件主要有以下几种：

（1）manager 日志

manager 日志为 Manager 运行过程中产生的日志。

（2）logcat 日志

logcat 日志为设备运行过程中产生的所有日志，对于任何操作都会在 logcat 中产生相应日志信息。

（3）Fuzzer 运行日志

Fuzzer 运行日志为 Fuzzer 程序在设备中运行产生的日志，Fuzzer 在运行过程中可能会产生错误，因此也需要记录其日志来分析其错误原因。

（4）tombstone 日志

当 Fuzzer 程序运行过程中，系统会注册多个信号处理程序，如果 native 的 C+代码出现崩溃，这些处理程序会将一份基本崩溃转储信息写入 logcat，并将一个更详细的 Tombstone 文件写入/data/tombstones/中。Tombstone 是一个包含与崩溃进程相关的额外数据的文件。具体而言，该文件包含崩溃进程中所有线程（而不只是捕捉到信号的线程）的堆栈轨迹、完整的内存映射，以及所有打开的文件描述符的列表。

tombstone 日志也是本文需要分析的主要日志文件，在模糊测试过程中，设备会不断地发生崩溃，可能是由于 HWASn 检测到内存错误而导致崩溃，也可能是 C++代码逻辑错误而导致崩溃；该日志文件正好记录了其崩溃之前的各种信息：通过对该类文件的分析，本文在复现了部分谷歌已经认证的漏洞基础上，还发现诸多新漏洞，并且已提交至谷歌。还详细统计情况如下表 3-1 所示。

表 3-1 发现漏洞统计列表

漏洞产生的库或进程	系统服务	漏洞类型	发现情况	主要利用方式
libcutils.so	native_handle	null pointer dereference	复现	拒绝服务攻击
Libaaudioservice.so	IAAudioService	null pointer dereference	复现	拒绝服务攻击
libmediametrics.so	IMediaAnalyticsService	null pointer dereference	新发现	拒绝服务攻击
libsurfaceflinger.so	SafeInterface	invalid memory access	新发现	拒绝服务攻击
libsensorservice.so	SensorService	illegal FileDescriptor	复现	越权访问资源
libsurfaceflinger.so	SurfaceFlinger	invalid memory access	复现	拒绝服务攻击
libmediadrm.so	IDrm	new_capacity overflow	新发现	内存溢出攻击
libmediadrm.so	ICrypto	new_capacity overflow	复现	内存溢出攻击
libmediadrm.so	ICrypto	null pointer dereference	复现	拒绝服务攻击
statsd	StatsService	heap-buffer-overflow	新发现	堆内存溢出攻击
statsd	StatsService	null pointer dereference	复现	拒绝服务攻击
installd	InstalldNativeService	stack-buffer-overflow	新发现	栈内存溢出攻击
incidentd	IncidentService	null pointer dereference	复现	拒绝服务攻击
gatekeeperd	GateKeeperService	CHECK failure	新发现	拒绝服务攻击
mediaserver	MediaPlayerService	CHECK failure	新发现	拒绝服务攻击
libwificond.so	WifiScannerImpl	CHECK failure	新发现	拒绝服务攻击
netmgrd		global-overflow	新发现	全局变量溢出攻击
netmgrd		heap-buffer-overflow	新发现	堆内存溢出攻击
audioserver	AudioPolicyService	Assertion failed	新发现	拒绝服务攻击
installd	InstalldNativeService	Abort Signal Error	新发现	拒绝服务攻击
installd	InstalldNativeService	CHECK failure	新发现	拒绝服务攻击

3.5 案例分析

3.5.1 漏洞 1 (new_capacity overflow)

在分析崩溃日志时，发现了一系列包含以下信息的 tombstone 文件，这些崩溃是由于"new_capacity overflow"引起的。通过对图 3-3 所示的调用栈信息和 IDrm 系统服务源码的分析，发现所有崩溃都是由 BnDrm:readVector 函数触发的。该函数的源码如图 3-4 所示，**其功能是从 Fuzzer 构造的测试用例中读取一个 vector 数组。**在函数内部，会调用 insertAt 函数为该数组分配一个缓冲区，缓冲区的大小取决于 Parcel data 对象中的 size 变量。同时，insertAt 函数会对插入位置进行合法性校验，如果插入索引不满足条件，会直接返回"BAD INDEX"。然而，insertAt 函数没有检查 size 的值，而 Fuzzer 可以随意构造 size 的值，例如使用负数作为 size 的值，从而触发漏洞。这个漏洞会导致 Drm 系统服务无法为用户提供服务，攻击者可以利用该漏洞进行拒绝服务攻击。

换言之，通过分析调用栈和源代码，我们可以确定造成崩溃的原因是 BnDrm:readVector 函数中的漏洞。该漏洞源于对 size 变量的不正确处理，导致 insertAt 函数分配缓冲区时出现溢出情况。攻击者可以通过构造恶意的测试用例来利用这个漏洞，从而使 Drm 系统服务无法正常工作，实施拒绝服务攻击。这个问题需要在 IDrm 系统服务源码中进行修复，确保在 insertAt 函数中对 size 变量进行合法性检查，防止出现溢出的情况。这样可以提高系统的安全性和稳定性，防止恶意攻击。

```
Build fingerprint: 'Android/aosp_taimen/taimen:9/PQ3A.190801.002/root02252256:userdebug/test-keys'
Revision: 'rev_10'
ABI: 'arm'
pid: 938, tid: 1054, name: Binder:938_1  >>> /system/bin/mediadrmserver <<<
signal 6 (SIGABRT), code -6 (SI_TKILL), fault addr --------
Abort message: 'new_capacity overflow'
    r0  00000000  r1  0000041e  r2  00000006  r3  00000008
    r4  000003aa  r5  0000041e  r6  d0bfe034  r7  0000010c
    r8  d602e6a9  r9  00000000  r10 d2718000  r11 d602e6a9
    ip  d0bfe4b8  sp  d0bfe020  lr  f34adfe9  pc  f34a5e36

backtrace:
    #00 pc 0001ce36  /system/lib/libc.so (abort+58)
    #01 pc 00006ccd  /system/lib/liblog.so (__android_log_assert+156)
    #02 pc 00016ca1  /data/asan/system/lib/libutils.so (android::VectorImpl::_grow(unsigned int, unsigned int)+1692)
    #03 pc 00016e1f  /data/asan/system/lib/libutils.so (android::VectorImpl::insertAt(unsigned int, unsigned int)+50)
    #04 pc 00018e69  /data/asan/system/lib/libmediadrm.so (android::BnDrm::readVector(android::Parcel const&, android::Vector<unsigned char>&) const+28)
    #05 pc 00019e35  /data/asan/system/lib/libmediadrm.so (android::BnDrm::onTransact(unsigned int, android::Parcel const&, android::Parcel*, unsigned int)+3964)
    #06 pc 0003aff5  /data/asan/system/lib/libbinder.so (android::BBinder::transact(unsigned int, android::Parcel const&, android::Parcel*, unsigned int)+116)
    #07 pc 00056cd9  /data/asan/system/lib/libbinder.so (android::IPCThreadState::executeCommand(int)+1088)
    #08 pc 0005659d  /data/asan/system/lib/libbinder.so (android::IPCThreadState::getAndExecuteCommand()+252)
    #09 pc 000577b3  /data/asan/system/lib/libbinder.so (android::IPCThreadState::joinThreadPool(bool)+54)
    #10 pc 000b71a1  /data/asan/system/lib/libbinder.so (android::PoolThread::threadLoop()+32)
    #11 pc 00012e27  /data/asan/system/lib/libutils.so (android::Thread::_threadLoop(void*)+626)
    #12 pc 00063679  /system/lib/libc.so (__pthread_start(void*)+22)
    #13 pc 0001dff9  /system/lib/libc.so (__start_thread+24)
```

图 3- 3　IDrm 系统服务崩溃错误信息及调用栈

```
1.  void BnDrm::readVector(const Parcel &data,Vector<uint8_t>&vector)const{
2.      uint32_t size=data.readInt32();
3.      if (vector.insertAt((size_t)0,size)<0){
4.          vector.clear():
5.      }
6       ...
7.  }
8.   ...
9.  ssize_t VectorImpl:insertAt(size_t index,size_t numItems)
10. {
11.     return insertAt(0,index,numItems);
12. }
13. ...
14. ssize_t VectorImpl::insertAt(const void*item,size_t index,size_t numItems)
15. {
16.     if (index >size())
17.         return BAD INDEX;
18.     ...
19. }
```

图 3- 4 readVector 源码

3.5.2 漏洞 2 (heap-buffer-overflow)

在 tombstone 日志文件中，发现如下图 3-5 所示的所示调用栈信息，并且 HWASan 报出了 heap-buffer-overflow 错误，其调用栈如图 3-5 所示：

```
backtrace:
    #07 pc 00000000000ac83c  /system/bin/statsd (android::os::statsd::UidMap::updateMap(long const&, std::__1::vector<int, std::__1::allocator<int>> const&,
std::__1::vector<long, std::__1::allocator<long>> const&, std::__1::vector<android::String16, std::__1::allocator<android::String16>> const&)+812)
    #08 pc 00000000000cbde8  /system/bin/statsd (android::os::statsd::StatsService::informAllUidData(std::__1::vector<int, std::__1::allocator<int>> const&,
std::__1::vector<long, std::__1::allocator<long>> const&, std::__1::vector<android::String16, std::__1::allocator<android::String16>> const&)+296)
    #09 pc 000000000012c204  /system/bin/statsd (android::os::BnStatsManager::onTransact(unsigned int, android::Parcel const&, android::Parcel*, unsigned int)+3784)
    #10 pc 00000000000c7c64  /system/bin/statsd (android::os::statsd::StatsService::onTransact(unsigned int, android::Parcel const&, android::Parcel*, unsigned int)
+1016)
    #11 pc 0000000000055938  /data/asan/system/lib64/libbinder.so (android::BBinder::transact(unsigned int, android::Parcel const&, android::Parcel*, unsigned int)+200)
    #12 pc 0000000000072ad8  /data/asan/system/lib64/libbinder.so (android::IPCThreadState::executeCommand(int)+1312)
    #13 pc 0000000000072390  /data/asan/system/lib64/libbinder.so (android::IPCThreadState::getAndExecuteCommand()+320)
    #14 pc 00000000000735c4  /data/asan/system/lib64/libbinder.so (android::IPCThreadState::joinThreadPool(bool)+60)
    #15 pc 00000000000cf790  /data/asan/system/lib64/libbinder.so (android::PoolThread::threadLoop()+56)
    #16 pc 000000000000168a4  /data/asan/system/lib64/libutils.so (android::Thread::_threadLoop(void*)+804)
    #17 pc 0000000000083114  /system/lib64/libc.so (__pthread_start(void*)+36)
    #18 pc 000000000000233bc  /system/lib64/libc.so (__start_thread+68)
```

图 3- 5 StatsService 系统服务崩溃调用栈

根据调用栈，该崩溃发生在由 AIDL 动态生成的 Native 系统服务 StatsService 中，并且该系统服务为 statsd 守护进程，通过对源码进行分析发现其崩溃发生在 Call::NFORMALLUIDDATA 事务中，该事务会从 data 对象中反序列化三个数组，分别为：

(1)vector<int32 t>in uid

(2)vector<int64 t>in_version

(3)vector<::android::String16>in_app

其调用路径如图 3-6 所示：

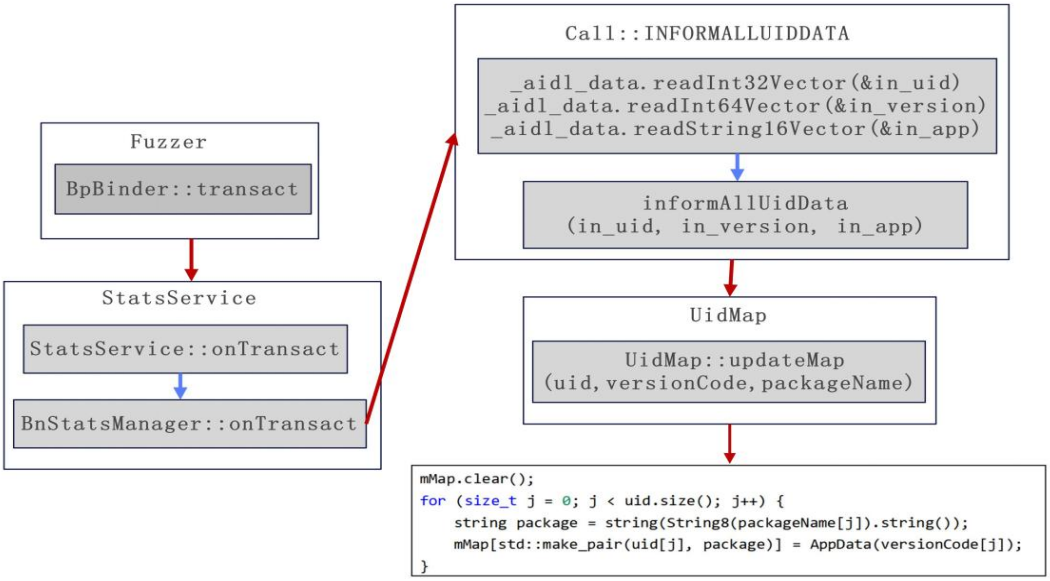


图 3- 6 StatsService 系统服务崩溃调用路径

这三个向量会传递到 informAllUidData 函数中，然后进一步转发到 UidMap:updateMap 函数。在 updateMap 函数中，会对这三个向量进行遍历。这三个向量位于同一个 for 循环中，并以数组 d 的大小作为循环次数。根据设计逻辑，这三个向量应该具有相同的长度，以确保它们之间的元素是一一对应的，从而使迭代过程正常进行。

然而，通过 Fuzzer 生成的测试用例，可以产生比其他两个向量更长的 uid 向量，从而构造出不正常的数据。当另外两个向量访问超出其分配的内存空间索引时，就会发生 heap-buffer-overflow，导致系统崩溃。

在 Fuzzer 的实现中，对于数组类型的构造，首先会生成一个数组的大小，然后生成相应数量的元素。因此，完全有可能生成的三个数组的大小不相等，这就会导致系统崩溃。攻击者可以利用这个漏洞来发起拒绝服务攻击。

换言之，由于 Fuzzer 生成的畸形数据，导致了三个向量的长度不一致。当在 updateMap 函数中遍历这三个向量时，超出边界的访问会触发 heap-buffer-overflow，导致系统崩溃。攻击者可以利用这个漏洞对系统发起拒绝服务攻击。

为了解决这个问题，需要对 updateMap 函数进行修复，确保三个向量具有相同的长度，以避免超出边界的访问。这样可以提高系统的安全性和稳定性，防止恶意攻击。

3.5.3 漏洞 3 (CHECK failure)

崩溃是由‘Check failed: mAudioEncoder == AUDIO_ENCODER_AAC || mAudioEncoder == AUDIO_ENCODER_HE_AAC || mAudioEncoder == AUDIO_ENCODER_AAC_ELD’引起的。如图 3-7 所示，通过对错误报告中的调用栈信息和 MediaServer 服务源码的分析，发现所有崩溃都是由 MediaRecorderClient::prepare() 函数触发的。

如图 3-8 该函数的源码所示，其功能是准备音频录制。在函数内部，它会首先检查 mRecorder 是否已经被初始化。若没有，函数将返回一个 NO_INIT 错误。否则，它将调用 mRecorder->prepare()。考虑到崩溃信息和调用栈中的 android::StagefrightRecorder::setupAACRecording() 函数，我们可以推断问题可能出现在 StagefrightRecorder 类的 prepare 或 setupAACRecording 函数中，特别是与 mAudioEncoder 相关的部分。

这里的关键是，mAudioEncoder 的值在准备阶段并未被正确设置，导致其不满足任何预期中的三个值。这可能是由于 mAudioEncoder 的值没有被正确地初始化，或者在某个时刻被错误地修改。

换言之，通过分析调用栈和源代码，我们可以确定造成崩溃的原因是 MediaRecorderClient::prepare() 函数中的漏洞。该漏洞源于对 mAudioEncoder 变量的不正确处理，导致在执行 setupAACRecording 时产生了一个失败的检查。攻击者可能会利用这个问题，构造特定的输入或请求，从而使 mediaserver 服务崩溃，实现拒绝服务攻击。

为了解决这个问题，建议在 StagefrightRecorder 类的源码中进行修复。确保在任何可能修改 mAudioEncoder 值的位置，都对其进行正确的检查和初始化，以防止这种崩溃再次发生。这样可以提高系统的安全性和稳定性，防止恶意攻击。

```
*** *** *** *** *** *** *** *** *** *** *** *** *** *** *** ***
Build fingerprint: 'Android/aosp_coral/coral:13/TP1A.220905.004/eng.root.20230622.191727:userdebug/test-keys'
Revision: 'MP1.0'
ABI: 'arm'
Timestamp: 2023-08-06 19:48:07.571227696+0800
Process uptime: 1239s
Cmdline: /system/bin/mediaserver
pid: 1303, tid: 3373, name: binder:1303_3  >>> /system/bin/mediaserver <<<
uid: 1013
signal 6 (SIGABRT), code -1 (SI_QUEUE), fault addr --------
Abort message: 'Check failed: mAudioEncoder == AUDIO_ENCODER_AAC || mAudioEncoder == AUDIO_ENCODER_HE_AAC || mAudioEncoder == AUDIO_ENCODER_AAC_ELD '
    r0  00000000  r1  00000d2d  r2  00000006  r3  f12e5e80
    r4  f12e5e90  r5  f12e5e78  r6  00000517  r7  0000016b
    r8  00000000  r9  ffffffff  r10 f12e5e80  r11 00000000
    ip  00000d2d  sp  f12e5e60  lr  f403bb97  pc  f403bbaa

backtrace:
      #00 pc 00039baa  /apex/com.android.runtime/lib/bionic/libc.so (abort+138) (BuildId: c794f967f88519cd1ea73c556ac74e73)
      #01 pc 0000490f  /system/lib/liblog.so (__android_log_default_aborter+6) (BuildId: a05171b91ef356eb7defede577402a7e)
      #02 pc 0007281b  /system/lib/libbinder.so (android::base::LogMessage::~LogMessage()+230) (BuildId: bfb72ca2e666e9eb2ce3f7327a7fa274)
      #03 pc 0007c305  /system/lib/libmediaplayerservice.so (android::StagefrightRecorder::setupAACRecording()+432) (BuildId: 6fa5d0d09081717d021170711fe2964c)
      #04 pc 00078635  /system/lib/libmediaplayerservice.so (android::StagefrightRecorder::prepare()+36) (BuildId: 6fa5d0d09081717d021170711fe2964c)
      #05 pc 0006f299  /system/lib/libmediaplayerservice.so (android::MediaRecorderClient::prepare()+56) (BuildId: 6fa5d0d09081717d021170711fe2964c)
      #06 pc 00045be9  /system/lib/libmedia.so (android::BnMediaRecorder::onTransact(unsigned int, android::Parcel const&, android::Parcel*, unsigned int)+2484) (BuildId:
a3f91eaa106b04a36317c67148bec2a8)
      #07 pc 0003a1eb  /system/lib/libbinder.so (android::BBinder::transact(unsigned int, android::Parcel const&, android::Parcel*, unsigned int)+210) (BuildId: bfb72ca2e666e9eb2ce3f7327a7fa274)
      #08 pc 00041979  /system/lib/libbinder.so (android::IPCThreadState::executeCommand(int)+604) (BuildId: bfb72ca2e666e9eb2ce3f7327a7fa274)
      #09 pc 00041683  /system/lib/libbinder.so (android::IPCThreadState::getAndExecuteCommand()+98) (BuildId: bfb72ca2e666e9eb2ce3f7327a7fa274)
      #10 pc 0004 1d31  /system/lib/libbinder.so (android::IPCThreadState::joinThreadPool(bool)+44) (BuildId: bfb72ca2e666e9eb2ce3f7327a7fa274)
      #11 pc 00061209  /system/lib/libbinder.so (android::PoolThread::threadLoop()+12) (BuildId: bfb72ca2e666e9eb2ce3f7327a7fa274)
      #12 pc 0000d789  /system/lib/libutils.so (android::Thread::_threadLoop(void*)+264) (BuildId: 56d8b221397e3843539513a676c6463b)
      #13 pc 00083ccf  /apex/com.android.runtime/lib/bionic/libc.so (__pthread_start(void*)+40) (BuildId: c794f967f88519cd1ea73c556ac74e73)
      #14 pc 0003ab4d  /apex/com.android.runtime/lib/bionic/libc.so (__start_thread+30) (BuildId: c794f967f88519cd1ea73c556ac74e73)
```

图 3- 7 CHECK failure 错误调用栈信息

```
1.  status_t MediaRecorderClient::setParameters(const String8& params) {
2.          ...
3.          if (mRecorder == NULL) {
4.              ALOGE("recorder is not initialized");
5.              return NO_INIT;
6.          }
7.          return mRecorder->setParameters(params);
8.      }
9.      status_t MediaRecorderClient::prepare(){
10.         ALOGV("prepare");
11.         Mutex::Autolock lock(mLock);
12.         if (mRecorder == NULL) {
13.             ALOGE("recorder is not initialized");
14.             return NO_INIT;
15.          }
16.         return mRecorder->prepare();
17.     }
```

图 3- 8 MediaRecorderClient 源码

3.5.4 漏洞 4 (CHECK failure)

在分析 crash 崩溃日志时，如图 3-9 所示，发现在 LibcurlHttpFetcher::ResumeTransfer 中的某个条件检查（CHECK）失败，会导致程序终止。

从给出的调用栈回溯信息可以判断，此错误源于程序中的一个断言（assert）

失败。断言是程序员放在代码中的一种保护机制，用来检查某个预期为真的条件是否真的为真。如果该条件为假，程序会以错误或异常的方式终止，防止继续执行可能产生错误结果的代码。源码如图 3-10 所示。

断言失败通常是因为编程错误或数据错误。如果断言是由于数据错误引起的，那么这可能表明存在一个更深层次的问题，如数据破损、并发错误或者恶意攻击。

这个漏洞会导致 update engine 服务无法正常工作，进而影响到设备的更新。攻击者可以控制引发断言失败的数据来阻止设备接收到重要的安全更新，或者引导设备安装恶意的更新。

```
backtrace:
    #00 pc 000000000005ca24  /apex/com.android.runtime/lib64/bionic/libc.so (abort+312) (BuildId: 89cf4062a3075d8b4285c9822b287424)
    #01 pc 00000000000077e4  /system/lib64/liblog.so (__android_log_default_aborter+12) (BuildId: 71ac7a1a095cdabe88d458bea570d7a8)
    #02 pc 00000000000d80b4  /system/lib64/libbinder.so (android::base::LogMessage::~LogMessage()+672) (BuildId: dad9a459b698f3ffa7a5604fb8f52c7b)
    #03 pc 0000000000007150  /system/bin/gatekeeperd (android::GateKeeperProxy::adjust_userId(unsigned int)+312) (BuildId: 7f2a59e1e0e5151340adfd59a8e53db2)
    #04 pc 000000000000628c  /system/bin/gatekeeperd (android::GateKeeperProxy::clearSecureUserId(int)+272) (BuildId: 7f2a59e1e0e5151340adfd59a8e53db2)
    #05 pc 0000000000007528  /system/lib64/libgatekeeper_aidl.so (android::service::gatekeeper::BnGateKeeperService::onTransact(unsigned int, android::Parcel const&, android::Parcel*, unsigned int)+2296) (BuildId:
fd926f3e580951bd7a6aa6e549076855)
    #06 pc 000000000056a9c  /system/lib64/libbinder.so (android::BBinder::transact(unsigned int, android::Parcel const&, android::Parcel*, unsigned int)+312) (BuildId: dad9a459b698f3ffa7a5604fb8f52c7b)
    #07 pc 0000000000069a24  /system/lib64/libbinder.so (android::IPCThreadState::executeCommand(int)+1472) (BuildId: dad9a459b698f3ffa7a5604fb8f52c7b)
    #08 pc 0000000000069334  /system/lib64/libbinder.so (android::IPCThreadState::getAndExecuteCommand()+232) (BuildId: dad9a459b698f3ffa7a5604fb8f52c7b)
    #09 pc 000000000006a01c  /system/lib64/libbinder.so (android::IPCThreadState::joinThreadPool(bool)+88) (BuildId: dad9a459b698f3ffa7a5604fb8f52c7b)
    #10 pc 00000000000043b4  /system/bin/gatekeeperd (main+864) (BuildId: 7f2a59e1e0e5151340adfd59a8e53db2)
    #11 pc 0000000000052810  /apex/com.android.runtime/lib64/bionic/libc.so (__libc_init+136) (BuildId: 89cf4062a3075d8b4285c9822b287424)
```

图 3- 9 CHECK failure 错误调用栈信息

```
1.  Status clearSecureUserId(int32_t userId) override {
2.      IPCThreadState* ipc = IPCThreadState::self();
3.      const int calling_pid = ipc->getCallingPid();
4.      const int calling_uid = ipc->getCallingUid();
5.      if (!PermissionCache::checkPermission(KEYGUARD_PERMISSION,
6.      calling_pid, calling_uid)) {
7.          ALOGE("%s: permission denied for [%d:%d]",
8.          __func__, calling_pid, calling_uid);
9.          return Status::ok();
10.     }
11.     clear_sid(userId);
12.     if (hw_device) {
13.         uint32_t hw_userId = adjust_userId(userId);
14.         hw_device->deleteUser(hw_userId, [](const GatekeeperResponse&) {});
15.     }
16.     return Status::ok();
17. }
```

图 3- 10 clearSecureUserId 源码

3.6 小结

本章介绍了本作品的两大主要功能进行功能性测试的方案。第一部分是针对Native 系统服务接口收集与依赖推断功能的测试方案，通过设计测试用例、执行接口提取、验证提取结果和边界条件测试等步骤，测试该功能的准确性、完整性和可靠性。第二部分是针对事务接口模型提取功能的测试方案，包括定义测试目标和范围、设计测试用例、执行接口模型提取、验证提取结果和边界条件测试等步骤。通过对这两个功能进行测试，得出了功能测试结果和分析。对于 Native 系统服务接口提取与依赖推断功能，成功收集了 118 个系统服务并生成了依赖推断图。对于事务接口模型提取功能，成功提取了 1313 个接口模型，并分析了提取过程中利用的技术手段和信息。

在测试过程中，发现了多个漏洞案例。其中，漏洞 1 是由于 Native 系统服务中的接口函数处理不正确导致的拒绝服务漏洞，攻击者可以利用该漏洞发起拒绝服务攻击。而漏洞 2 是由于向量长度不一致导致的堆溢出漏洞，攻击者可以通过构造恶意测试用例触发该漏洞，进而导致系统崩溃。漏洞 3 是由于 MediaRecorderClient::prepare() 函数中的一个错误引起的，具体涉及 mAudioEncoder 变量的不正确处理。这个错误使攻击者可以利用 mediaserver 服务产生拒绝服务攻击。漏洞 4 是由于程序中的断言失败导致的，攻击者可以利用该漏洞阻止设备接收重要的安全更新或安装恶意更新。对于这些漏洞，建议修复相应的代码逻辑和错误处理，以提高系统的安全性和稳定性，防止恶意攻击。通过对崩溃日志的分析，可以发现并解决潜在的安全问题，进一步提升系统的质量和可靠性。

第四章 创新性说明

本作品的研究角度新颖，实现技术巧妙、实操效果良好。**我们在调研中发现目前在国内外，对安卓原生系统漏洞的研究仍然不足，缺乏一种既有效又可行的检测工具，而我们的产品正是为解决这一问题而设计的。我们的产品不仅具有版本普适性，更重要的是，其实现技术巧妙，实操效果良好，有着极高的实用价值。**本作品能够有效填补该领域空白，推动相关研究进展和行业良性发展。

4.1 研究方向开创性

当今，针对 Android 系统服务安全的研究主要聚焦在应用程序安全、网络安全和数据安全三个方面。虽然各自有一定的深度，但由于方向分散，没有形成全面的视角和策略。同时，这些研究的适用范围相对有限，无法覆盖所有可能的安全隐患。**相对而言，针对 Android 原生系统服务的安全研究则被普遍忽视，尽管这是一个充满潜力且具有极高价值的研究方向。这不仅涉及到整个平台的安全性，而且在很大程度上也影响着网络安全和数据安全，值得我们高度重视。**

通过我们的调研，近年相关数据库收录的 Android 系统安全相关论文中，仅有极少数专门针对 Android 原生系统服务安全展开探讨。更重要的是，这些现有研究在模糊测试应用方面明显存在限制和不足，没有形成一种针对 Android 原生系统服务的全方位、深入的测试方案。这一现象揭示了当前许多研究者和开发者对于 Android 原生系统服务安全重要性的认识缺失，以及对可能引发严重安全隐患的领域的忽略。

本作品旨在填补这一研究空白，创新性地提出并实施了一套针对 Android 原生系统服务的模糊测试方案。**这一方案首次实现了对 Android 原生系统服务的全面和深入的模糊测试，旨在为解决 Android 原生系统服务的安全问题提供一种全新的工具和方法。**此项工作不仅可能推动该方向的学术进展，而且有潜力引领一个崭新的研究方向，激发更多关于 Android 原生系统服务安全的探讨和研究，从而增强整个 Android 生态系统的安全性。

4.2 技术实现巧妙性

（1）该作品在服务接口文件收集阶段，通过对 Android Open Source Project(AOSP)源码进行深入分析和编译，并记录编译命令。这样做可以精确地收集所有与 Android Native 系统服务相关的文件，**提升了测试的全面性和准确性。**

（2）本作品通过接口模型预处理和后处理，**采用了基于 Clang Abstract Syntax Tree (Clang AST)的技术来提取和解析服务接口的树形结构。这个处理过程非常巧妙，通过使用 AST，可以更准确、高效地理解和分析服务接口的结构和语义。**

（3）在依赖推断阶段，本作品通过提取的接口模型，推断出服务接口之间的依赖，这样可以更好地理解和模拟系统服务的实际运行状态，从而提高模糊测试的效果。

（4）在 Fuzzer 设计与实现上，根据接口模型和依赖关系生成测试用例，充分考虑了约束和依赖关系，使测试更加贴近真实场景。**而且，实现了 Manager 模块以自动运行和监控设备状态，保证了测试的稳定性和有效性。**

本作品在漏洞挖掘覆盖率和多版本适用性与易调整性方面表现出了其优势。且具有自动提取完整服务接口、自动化测试与监控日志同步和针对 Android 最新的版本 13 等特点。深度探索了 Android 原生系统服务之间的接口关系，实现全流程自动化检测，有效降低人工成本和避免误差。并且对于不同的 Android 版本，可以灵活调整测试策略和测试用例，具有较强的适用性。

4.3 系统实战可行性

（1）**本作品在开发完毕后的实际操作中，可用性强，嗅探感知度高。实现过程中我们成功收集了 118 个 Android Native 系统服务的相关文件，包括 41 个实名服务和 77 个匿名服务，并成功提取了 1313 个事务的接口模型。**在 Pixel4xl 的 Android 13 版本上进行了实际测试，有效产生了大量的崩溃日志，进一步分析这些日志，我们详细解析了两个具体漏洞案例，验证了本模糊测试工具在 Android 13 版本下的有效性。以上实操数据足以证明，我们的测试方案和工具确实可行，并具有良好的实战性。

（2）本作品使用前景广泛，我们的作品在商业市场上有着广阔的应用前景。由于 Android 系统开源性的特点，许多移动终端厂商、用户和应用开发商都在使用 Android 系统，并对其进行了各种定制和修改。**对于不同的厂商，我们的模糊测试方案可以根据各自的具体需求进行定制化修改，适应各自的硬件和功能需求，对各种定制化的 Android 系统进行有效的漏洞检测。**这种灵活性和适应性，使我们的作品能够适应广阔的商业市场，满足各种不同厂商对系统服务的安全性和稳定性的要求。

4.4 小结

本作品研究了 Android 原生系统服务安全，通过开创性的模糊测试方法，填补了国内外在该领域的研究空白。其技术实现巧妙，利用 Clang AST 技术解析接口模型，并考虑了漏洞挖掘覆盖率、自动化提取与检测以及多版本适用性与易调整性。在实际操作中，作品表现出较高的可行性和实战性，成功收集 Android Native 系统服务相关文件并在 Android 13 版本进行测试。其商业市场应用前景广阔，可以根据不同厂商需求定制化修改，满足定制化 Android 系统的安全检测需求。综上所述，本作品具有重要的研究意义和实用价值，对 Android 原生系统服务安全的推动和整体生态系统的提升具有积极影响。

第五章 总结与展望

5.1 总结

Android 系统服务为用户应用提高了最基本的支持及核心功能，根据 Google 近些年发布的安全公告，仅在系统服务中就已经发现了数百个安全漏洞，Android Native 系统服务是系统服务的最底层实现，由于使用 C++编写，有大量危害严重的安全漏洞存在于 Native 系统服务中，因此有必要对 Android Native 系统服务进行严格有效的安全测试。

表 5-1 报错日志漏洞信息表

漏洞产生的库或进程	系统服务	漏洞类型	所处版本	主要利用方式	发现情况
libcutils.so	native_handle	null pointer dereference	Android 9	拒绝服务攻击	复现
Libaaudioservice.so	IAAudioService	null pointer dereference	Android 10	拒绝服务攻击	复现
Libmediametrics.so	IMediaAnalyticsService	null pointer dereference	Android 13	拒绝服务攻击	新发现
Libsurfaceflinger.so	SafeInterface	invalid memory access	Android 13	拒绝服务攻击	新发现
Libsensorservice.so	SensorService	illegal FileDescriptor	Android 10	越权访问资源	复现
Libsurfaceflinger.so	SurfaceFlinger	invalid memory access	Android 9	拒绝服务攻击	复现
libmediadrm.so	IDrm	new_capacity overflow	Android 13	内存溢出攻击	新发现
libmediadrm.so	ICrypto	new_capacity overflow	Android 10	内存溢出攻击	复现
libmediadrm.so	ICrypto	null pointer dereference	Android 10	拒绝服务攻击	复现
statsd	StatsService	heap-buffer-overflow	Android 13	堆内存溢出攻击	新发现
statsd	StatsService	null pointer dereference	Android 10	拒绝服务攻击	复现
installd	InstalldNativeService	stack-buffer-overflow	Android 13	栈内存溢出攻击	新发现
incidentd	IncidentService	null pointer dereference	Android 10	拒绝服务攻击	复现
gatekeeperd	GateKeeperService	CHECK failure	Android 13	拒绝服务攻击	新发现
mediaserver	MediaPlayerService	CHECK failure	Android 13	拒绝服务攻击	新发现
libwificond.so	WifiScannerImpl	CHECK failure	Android 13	拒绝服务攻击	新发现
netmgrd		global-overflow	Android 13	全局变量溢出攻击	新发现
netmgrd		heap-buffer-overflow	Android 13	堆内存溢出攻击	新发现
audioserver	AudioPolicyService	Assertion failed	Android 13	拒绝服务攻击	新发现
installd	InstalldNativeService	Abort Signal Error	Android 13	拒绝服务攻击	新发现
installd	InstalldNativeService	CHECK failure	Android 13	拒绝服务攻击	新发现

本作品的研究角度新颖，实现技术巧妙、实操效果良好。我们在调研中发现目前关于安卓原生系统服务漏洞的安全研究较为匮乏，国内外尚无关于有效可行且具普适性的检测工具。**本作品能够有效填补该领域空白，推动相关研究进展和行业良性发展。**

5.2 展望

本作品实现的对 Android Native 系统服务的模糊测试仍有一些可以改进的方面，将会在今后的研究中进行，包括以下几个方面:

(1)本作品中在 Service 端的 onTransact 函数中提取接口模型时，没有将复杂的杂项数调用考虑到其中，比如一个 Parcel 类的 data 对象可能会传入该函数中进行读取输入数据或者写入输出数据，**这部分杂项函数相较于其他变量提取较为复杂和特殊，后续研究会对这一部分进行特殊处理。**

(2)在对服务接口进行依赖推断时，本文还发现了 60 个用户自定义服务接口，在对系统服务进行测试时，可能需要使用这些接口，因此需要这些接口服务需要由用户自己定义和实现，由于对 AOSP 源码知识有限，有些复杂的自定义接口还未实现，因此在模糊测试过程中可能会由于自定义服务接口获取不到而出现异常情况，在后续研究会继续加强对源码知识的解读，**实现更多的用户自定义服务接口。**

(3)在本文对 Android Native 系统服务的模糊测试并未使用到代码覆盖率引导，而是根据提取的事务接口模型来进行随机模糊测试，**因此后续工作方案将考虑将 AFL 模糊测试工具移植到本作品中进行结合使用，并且也会针对性地应用在未来 Android 更高版本新增的 Native 系统服务中。同时也会考虑与不同类型的模糊测试工具进行相互配合，通过组合运行，提高测试的全面性和覆盖范围。**

参考文献

[1] Liu B, Zhang C, Gong G, et al. {FANS}: Fuzzing android native system services via automated interface analysis[C]//29th USENIX Security Symposium (USENIX Security 20). 2020: 307-323.Serebryany K,Bruening D,Potapenko A,et al.Addresssanitizer:A fast address sanity checker[J].

[2] 张玉清，王凯，杨欢，等.Android 安全综述 U.计算机研究与发展 2014,51(7):1385-1396.

[3] AppBrain.NumberofavailableAndroidapplications[EB/OL].(2023-03-12)[2023-03-18].http://www.appbrain.com/stats.

[4] 邹圳，周安民.Android 系统服务漏洞挖掘技术研究。现代计算机，2019(13)90-95.

[5] 张志威，甘刚.基于遗传算法的 Android 系统服务漏洞挖掘 U.计算机与现代化 2020(08):114121.

[6] Cao C,Gao N,Liu P,et al.Towards analyzing the input validation vulnerabilities associated with android system services[C]//Proceedings of the 31st Annual Computer Security Applications Confer cnce.2015:361-370.

[7] Feng H,Shin K G.Bindercracker.Assessing the robustness of android system services[J].arXiv preprint arXiv:1604.06964,2016.

[8] Android 开源项目。Android 安全公告[EB/OL].(2023-03-16)[2023-03-18].https:/source.android.com/docs/security/bulletin/asb-overview.

[9] 李红辉，齐佳，刘峰，等.模糊测试技术研究 U.中国科学：信息科学，2014,4410):1305-1322.

[10] 360 安全.《2022 年度中国手机安全状况报告》[EB/OL].(2022-12)[2023-03-18].https://pop.shouji.3

[11] Iannillo A K,Natella R,Cotroneo D,et al.Chizpurfle:A gray-box android fuzzer for vendor service c

[12] ustomizations[C]//2017 IEEE 28th International Symposium on Software Reliability Engineering (ISSRE).IEEE,2017:1-11.

[13] 张红.基于 Clang 的 AST 提取 结构体数据库插件的实现[J].电脑知识与技术,2017,13(06):19-21.DOI:10.1 4 004/j.cnki.ckt.2017.047.

[14] Android 开源项目.下载源代码[EB/OL].(2022-09-24)2023-03-16)https:/source.android.com/hl=Zh-cn.

[15] Android 源码查询.9.0.0r3EB/OL].2018-08-12)2023-03-l6]http // androidxref.com/9.0.0r3/.

[16] 王麒麟.Android 系统服务半自动化模糊测试工具研究与实现 D]北京：北京邮电大学，2020.

[17] Android 开源项目.调试 Android 平台原始代码 EB/OL.(2022-09-23)2023-03-l

8]https:/source..android.com/docs/core/tests/debug?hl=zh-cn.
[18] 何远，张玉清，张光华.基于黑盒遗传算法的 Android 驱动漏洞挖掘［J］.计算机学报,2017,40(5):1031-1043.
[19] 廖祥森.定制 Android 系统服务测试技术研究[D].南京：南京大学，2021.
[20] Fratantonio Y,Bianchi A,Robertson W,et al.TriggerScope:Towards detecting logic bombs in Android applications［C］‖ Proc of the IEEE Symposium on Security and Privacy,2016:377-396.
[21] Android Developers. "Android Studio." https://developer.android.com/studio/.
[22] 2012.Wang X,Yang Y,Zhu S.Automated hybrid analysis of android malwar ethr ough augmenting fuzzing with forced execution［J］.IEEE Transactios on Mobile Computing,2019,18(12):2768-2782.
[23] Wang Kai, Liu Qi-xu, Zhang Yu-qing. Android inter- application communi cation vulnerability mining technique based on fuzzing［J］.Journal of Uni versity of Chinese Aca- demy of Sciences,2014,31(6):827-835.(in Chinese)
[24] Zhang Mi,Yang Li,Zhang Jun-wei.FuzzerAPP：The robustness test of appli cation component communication in Android［J］.Journal of Computer Re search and Development,2017,54(2):338-347.(in Chinese)
[25] Zhao Sai,Liu Hao,Wang Yu-feng,et al.Fuzz testing of Android inter-compo nent communication［J］.Computer Science,2020,47(S2):303-309.(in Chines e)
[26] He Yuan,Zhang Yu-qing,Zhang Guang-hua.Android driver vulnerability disc overy based on black-box genetic algorithm［J］.Chinese Journal of Comp uters,2017,40(5):1031-1043.(in Chinese)
[27] Din F,Zamli K.Fuzzy adaptive teaching learning-based optimization strateg y for GUI functional test cases generation［C］‖ Proc of the 7th Internati onal Conference on Software and Computer Applications,2018:92-96.
[28] Zhang Xing,Feng Chao,Lei Jing,et al.Real time idle state decection method in fuzzing test in GUI program［J］.Journal of Software,2018,29(5):1288-1 302.(in Chinese)
[29] Zalewski M. American fuzzy lop ［CP/OL］.［2021-08-14］. https://lcamt uf.coredump.cx/afl/.
[30] Huo Wei,Dai Ge,Shi Ji,et al.Browser fuzzing technology based on pattern-generation［J］.Journal of Software,2018,29(5):1275-1287.(in Chinese)
[31] Li Wei-ming, Zhang Ai-fang,Liu Jian-cai,et al.An automa- tic network prot ocol fuzz testing and vulnerability discover- ing method［J］.Chinese Jour nal of Computers,2011,34(2):242-255.(in Chinese)
[32] Ma R,Ren S M,Ma K,et al.Semi-valid fuzz testing case generation for stat eful network protocol［J］.Tsinghua Science and Technology,2017,22(5):45

8-468.
[33] Walz A, Sikora A.Exploiting dissent:Towards fuzzing-based differential bla ck-box testing of TLS implementations［J］.IEEE Transactions on Depend able and Secure Comput- ing,2020,17(2):278-291.
[34] Ma R,Wang D,Hu C,et al.Test data generation for stateful network protoc ol fuzzing using a rule-based state machine［J］.Tsinghua Science and Te chnology,2016,21(3):352-360.
[35] Tacliad F,Nguyen T,Gondree M.DoS exploitation of allen-bradley's legacy protocol through fuzz testing［C］‖ Proc of the 3rd Annual Industrial Con trol System Security Workshop,2017:24-31.
[36] Zhang Ya-feng,Hong Zheng,Wu Li-fa,et al.Protocol state based fuzzing me thod for industrial control protocol［J］.Computer Science,2017,44(5):132-1 40.(in Chinese)
[37] Kim S, Jo W, Shon T.A novel vulnerability analysis approach to generate fuzzing test case in industrial control systems［C］‖ Proc of the IEEE Inf ormation Technology,Networking,Electronic and Automation Control Confer ence,2016:566-570.
[38] Li H F,Wang S L,Zhang B, et al.Network protocol security testing based on fuzz［C］‖ Proc of the 4th International Conference on Computer Scie nce and Network Technology,2015:955-958.
[39] Wang W,Sun H,Zeng Q.SeededFuzz:Selecting and generat- ing seeds for d irected fuzzing［C］‖ Proc of the 10th International Symposium on Theor etical Aspects of Software Engineering,2016:49-56.
[40] Li Jia-li,Chen Yong-le,Li Zhi,et al.Mining RTSP protocol vulnerabilities ba sed on traversal of protocol state graph［J］.Computer Science,2018,45(9): 171-176.(in Chinese)
[41] Zhang Wei-yao,Zhang Lei,Mao Jian-ling,et al.An automated method of unk nown protocol fuzzing test［J］.Chinese Journal of Computers,2020,43(4):6 53-667.(in Chinese)
[42] Peng C, Rajan A.Automated test generation for OpenCL kernels using fuz zing and constraint solving［C］‖ Proc of the 13th Annual Workshop on General Purpose Processing using Graphics,2020:61-70.
[43] Wang Ying,Wang Bing-qing,Guan Yong,et al.Differential fuzz testing of ro bot operating system［J］.Journal of Software,2021,32(6):1867-1881.(in Chi nese)
[44] Sha Le-tian, Xiao Fu,Yang Hong-ke,et al.Vulnerability discovery method f or virtualization in IaaS based on self-adapting fuzzing test［J］.Journal of Software,2018,29(5):1303-1317.(in Chinese)
[45] Gao J,Xu Y,Jiang Y,et al.EM-Fuzz:Augmented firmware fuzzing via memo

ry checking［J］.IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems,2020,39(11):3420-3432.
[46] Shukla A, Saidi S J,Schmid S,et al.Toward consistent SDNs:A case for network state fuzzing［J］.IEEE Transactions on Network and Service Management,2020,17(2):668-681.
[47] Zhang Q,Wang J,Gulzar M A,et al.BigFuzz:Efficient fuzz testing for data analytics using framework abstraction［C］‖ Proc of the 35th IEEE/ACM International Conference on Automated Software Engineering,2020:722-733.
[48] Li Wei-ming,Yu Jun-qing,Ai Shao-bo.PyFuzzer:Automatic in-memory fuzz testing method［J］.Journal on Communications,2013,34(S2):64-68.(in Chinese)
[49] Wang Ying,Gu Li-ze,Yang Yi-xian,et al.EWFT:Execution whitebox fuzzing for executables［J］.Chinese Journal of Electronics,2014,42(10):2016-2023.(in Chinese)
[50] Wen C,Wang H,Li Y,et al.MEMLOCK:Memory usage guided fuzzing［C］‖ Proc of the 42nd IEEE/ACM International Conference on Software Engineering,2020:765-777.
[51] Fu Y,Ren M,Ma F,et al.EVMFuzzer:Detect EVM vulnerabilities via fuzz testing［C］‖ Proc of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software E ngineering,2019:1110-1114.
[52] Kargén U,Shahmehri N.Turning programs against each other:High coverage fuzz-testing using binary-code mutation and dynamic slicing［C］‖ Proc of the 10th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering,2015:782-792.
[53] Le De-guang,Gong Sheng-rong,Wu Shao-gang,et al.Research on RTF array overflow vulnerability detection［J］.Journal on Communications,2017,38(5):96-107.(in Chinese)
[54] Lemieux C,Sen K.FairFuzz:A targeted mutation strategy for increasing greybox fuzz testing coverage［C］‖ Proc of the 33rd ACM/IEEE International Conference on Automated Software Engineering,2018:475-485.
[55] Rawat, S., Jain, V., Kumar, A., Cojocar, L., Giuffrida, C., & Bos, H. (2017). VUzzer: Application-aware evolutionary fuzzing. Proceedings of the Network and Distributed System Security Symposium (NDSS), 17. Retrieved from: https://www.ndss-symposium.org/ndss2017/.
[56] Su, T., Zheng, Q., Wang, K., & Li, W. (2016). Picking up my tab: Understanding and mitigating synchronized token lifting and spending in mobile payment. 25th USENIX Security Symposium (USENIX Security 16). Re

trieved from: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/su.

[57] Zhang, M., Duan, L., Yin, H., & Zhao, Z. (2017). System service call-oriented symbolic execution of Android Framework with application to vulnerability discovery and exploit generation. Proceedings of the 26th USENIX Security Symposium. Retrieved from: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/zhang.

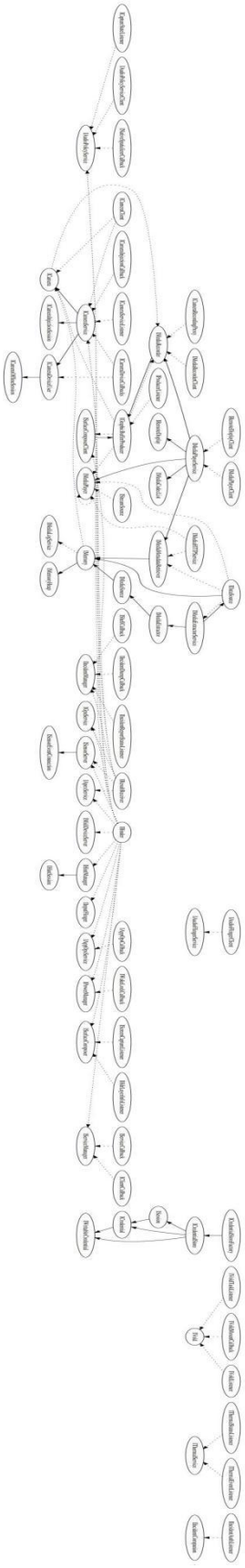
[58] Arzt, S., Rasthofer, S., Fritz, C., Bodden, E., Bartel, A., Klein, J., Le Traon, Y., Octeau, D., & McDaniel, P. (2014). Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. ACM SIGPLAN Notices, 49(6), 259-269. Retrieved from: https://dl.acm.or g/doi/10.1145/2666356.2594299.

[59] Rebert, A., Cha, S.K., Avgerinos, T., Foote, J., Warren, D., Grieco, G., & Brumley, D. (2014). Optimizing seed selection for fuzzing. Proceedings of the 2014 USENIX Conference on Offensive Technologies. Retrieved from: https://www.usenix.org/conference/woot14/workshop-program/presentation/rebert.

[60] H.D.T.Nguyen,D.Qi,A.Roychoudhury,and S.Chan-dra,"Semfix:Program repair via semantic analysis,"in Proceedings of the 35th International Conference on Software Engineering (ICSE).IEEE,2013,pp.772-781.

[61] G.Zhang,X.Zhou,Y.Luo,X.Wu,and E.Min,"PTfuzz:Guided Fuzzing With Processor Trace Feedback,"IEEE Access,vol.6,pp.37302-37313,2018.

附录 A　Native 系统服务接口依赖图



附录 B　作品总体框架大图

